

C++ Concurrency

& Threading — Interview Q&A

100 Expert Questions with Code Examples

- Thread Fundamentals: `std::thread`, `jthread`, `join`, `detach`, arguments
- Mutexes & Locking: `lock_guard`, `unique_lock`, `scoped_lock`, deadlock prevention
 - Condition Variables, Semaphores, Latches & Barriers (C++20)
- Atomic Operations & Memory Model: `memory_order`, happens-before, CAS
 - Thread Pool Design: work stealing, priority queues, graceful shutdown
- Thread-Safe Containers: `queue`, `stack`, sharded map, LRU cache, ring buffer
- Concurrency Pitfalls: data races, false sharing, priority inversion, ABA problem
 - Advanced Patterns: Active Object, RCU, Pipeline, Reactor, Actor Model
 - Diagnostics & Best Practices: TSan, profiling, structured concurrency
- Expert & Trading Systems: Lock-free MPMC, Disruptor, hazard pointers, NUMA, `futex`

C++11 C++14 C++17 C++20 C++23

SECTION 1 — Thread Fundamentals

Q1. What is `std::thread` and how do you create and join one? [Core]

▶ Answer

`std::thread` represents an OS-level thread. The constructor launches a thread executing the given callable (function, lambda, functor). You must either `join()` (wait for completion) or `detach()` (let it run independently) before the thread object is destroyed — otherwise `std::terminate()` is called. Always prefer RAII wrappers like `std::jthread` (C++20) to ensure automatic joining.

Example

```
#include <thread>
#include <iostream>

void workerFn(int id) {
    std::cout << "Thread " << id << " running\n";
}

int main() {
    // Launch thread with function
    std::thread t1(workerFn, 42);

    // Launch thread with lambda
    std::thread t2([](int x){ return x*2; }, 10);

    t1.join(); // wait for t1 to finish
    t2.join(); // wait for t2 to finish

    // WRONG: forgetting join/detach -> std::terminate()
    // std::thread t3(workerFn, 1); // dtor without join = crash!
}
```

Q2. What is the difference between `join()` and `detach()`? [Core]

▶ Answer

`join()` blocks the calling thread until the target thread finishes — the calling thread waits. `detach()` separates the thread from the `std::thread` object — the thread runs independently and the OS reclaims resources when it finishes. After `detach()`, you cannot join the thread or communicate with it safely. Detached threads are hard to synchronize and should be avoided in most cases. A `joinable()` check should precede any `join()` or `detach()` call.

Example

```
std::thread t([]{ doLongWork(); });

// join(): wait for completion
if (t.joinable()) t.join();

// detach(): fire and forget
std::thread t2([]{ backgroundTask(); });
t2.detach();
// t2 is no longer joinable -- we cannot wait for it

// DANGER with detach: if main() exits, detached threads
// accessing global/stack data get undefined behaviour

// SAFE pattern: RAII wrapper (C++20)
{
    std::jthread jt([]{ doWork(); });
} // joins automatically in destructor
```

Q3. How do you pass arguments to a thread and what are the pitfalls? [Core]**Answer**

Arguments are passed by value (copied) to the thread constructor by default — the thread gets its own copy. To pass by reference, wrap with `std::ref()` or `std::cref()`. Pitfalls: (1) Dangling references if the referenced object is destroyed before the thread finishes. (2) Passing a pointer to a local variable that goes out of scope. (3) Forgetting `std::ref` means a copy is modified, not the original.

Example

```
void increment(int& x) { x++; }

int val = 0;

// WRONG: passes copy – val not modified
// std::thread t(increment, val);

// CORRECT: pass by reference with std::ref
std::thread t(increment, std::ref(val));
t.join();
std::cout << val; // 1

// DANGER: dangling reference
std::thread* badThread = nullptr;
{
    int local = 5;
    badThread = new std::thread(increment, std::ref(local));
} // local destroyed here!
// badThread->join(); // UB: local already gone

// SAFE: capture by value in lambda
int safe = 42;
std::thread t2([safe]() mutable { safe++; }); // owns copy
t2.join();
```

Q4. What is `std::jthread` (C++20) and how does it improve on `std::thread`? [Core]**Answer**

`std::jthread` is a RAII thread that (1) automatically joins in its destructor (no more `terminate()`), (2) integrates with `std::stop_token` / `std::stop_source` for cooperative cancellation. The thread function optionally takes a `stop_token` as its first argument and can periodically check `stop_requested()`. This eliminates the most common threading bugs: forgotten joins and lack of cancellation.

Example

```
#include <thread>
#include <stop_token>

// Auto-join on scope exit
{
    std::jthread t([]{ doWork(); });
} // joins here -- no terminate()

// Cooperative cancellation
std::jthread worker([](std::stop_token st) {
    while (!st.stop_requested()) {
        processItem();
        std::this_thread::sleep_for(std::chrono::ms(10));
    }
}
std::cout << "Stopping gracefully\n";
```

```
});

std::this_thread::sleep_for(std::chrono::seconds(1));
worker.request_stop(); // signal cancellation
// destructor joins -- worker has already stopped

// Stop callbacks
std::stop_source ss;
std::stop_callback cb(ss.get_token(), []{ cleanup(); });
ss.request_stop(); // fires cb immediately
```

Q5. What is `std::this_thread` and what utilities does it provide? [Core]

▶ Answer

`std::this_thread` is a namespace providing utilities for the currently executing thread: `get_id()` returns the thread's unique ID; `sleep_for(duration)` suspends for a given duration; `sleep_until(time_point)` suspends until a specific time; `yield()` hints the scheduler to switch to another thread. These are essential for implementing timeouts, polling loops, and cooperative scheduling.

Example

```
#include <thread>
#include <chrono>

using namespace std::chrono_literals;

// Get current thread ID
auto id = std::this_thread::get_id();
std::cout << "Thread ID: " << id << "\n";

// Sleep for duration
std::this_thread::sleep_for(100ms);

// Sleep until specific time
auto wakeAt = std::chrono::steady_clock::now() + 500ms;
std::this_thread::sleep_until(wakeAt);

// Yield: give up timeslice
while (!ready.load()) {
    std::this_thread::yield(); // avoids busy-spin
}

// Polling with backoff
for (int i = 0; !done.load(); ++i) {
    if (i < 10) std::this_thread::yield();
    else std::this_thread::sleep_for(1ms);
}
```

Q6. How do you return values from a thread? [Core]

▶ Answer

`std::thread` has no built-in return mechanism. Options: (1) Write to a shared variable protected by a mutex. (2) Use `std::promise/std::future` — the cleanest approach. (3) Use `std::packaged_task` which wraps a callable and connects it to a future. (4) Pass a reference parameter for the result. `std::async` returns a future directly, hiding thread management entirely.

Example

```
// Option 1: shared variable (requires sync)
int result = 0;
```

```

std::mutex m;
std::thread t([&]{ std::lock_guard lk(m); result = compute(); });
t.join();

// Option 2: promise/future (recommended)
std::promise<int> prom;
std::future<int> fut = prom.get_future();
std::thread t2([p=std::move(prom)]() mutable {
    p.set_value(compute()); // send result
});
t2.join();
std::cout << fut.get(); // receive result

// Option 3: std::async (simplest)
auto future = std::async(std::launch::async, []{ return compute(); });
std::cout << future.get(); // blocks until ready

// Option 4: packaged_task
std::packaged_task<int> task([]{ return compute(); });
auto fut2 = task.get_future();
std::thread t3(std::move(task));
t3.join();
std::cout << fut2.get();

```

Q7. What is `std::thread::hardware_concurrency()` and when should you use it? [Core]

Answer

`std::thread::hardware_concurrency()` returns a hint for the number of concurrent threads the hardware supports (typically number of CPU cores or logical processors). It returns 0 if not computable. Use it to set the default thread pool size. Do not use it blindly — actual optimal thread count depends on workload type (CPU-bound vs I/O-bound), memory, and cache effects.

Example

```

unsigned int cores = std::thread::hardware_concurrency();
std::cout << "Logical processors: " << cores << "\n";

// Safe pool size with fallback
size_t poolSize = std::max(1u,
    std::thread::hardware_concurrency());

// CPU-bound: use hardware_concurrency
size_t cpuPool = cores;

// I/O-bound: more threads OK (they spend time waiting)
size_t ioPool = cores * 4;

// Mixed: some heuristic
size_t mixedPool = cores * 2;

// Create thread pool
std::vector<std::thread> pool;
pool.reserve(poolSize);
for (size_t i = 0; i < poolSize; ++i)
    pool.emplace_back(workerLoop);

```

SECTION 2 — Mutexes & Locking

Q8. What are the different mutex types and when should each be used? [Mutex]

Answer

`std::mutex`: basic, non-recursive, non-timed — the default choice. `std::recursive_mutex`: allows the same thread to lock multiple times (prefer refactoring). `std::timed_mutex`: adds `try_lock_for`/`try_lock_until` for timed acquisitions. `std::recursive_timed_mutex`: both recursive and timed. `std::shared_mutex` (C++17): reader-writer lock — multiple shared (read) locks or one exclusive (write) lock. `std::shared_timed_mutex`: adds timed operations to `shared_mutex`.

Example

```
// Basic mutex
std::mutex m;
{ std::lock_guard lk(m); /* critical section */ }

// Shared (reader-writer) mutex
std::shared_mutex rw;

// Reader: multiple can hold simultaneously
std::shared_lock<std::shared_mutex> readLk(rw);

// Writer: exclusive
std::unique_lock<std::shared_mutex> writeLk(rw);

// Timed mutex: try for up to 100ms
std::timed_mutex tm;
if (tm.try_lock_for(std::chrono::milliseconds(100))) {
    // got the lock
    tm.unlock();
} else {
    // timed out
}
```

Q9. What is the difference between `lock_guard`, `unique_lock`, and `scoped_lock`? [Mutex] [RAII]

Answer

`lock_guard<M>`: RAII lock, locks on construction, unlocks on destruction, cannot be manually controlled, not movable. `unique_lock<M>`: RAII lock with full control — supports `defer_lock`, `try_to_lock`, `adopt_lock`, manual lock/unlock, movable; required for condition variables. `scoped_lock` (C++17): locks multiple mutexes atomically using deadlock-avoidance; non-movable. Prefer `lock_guard` for simplicity, `unique_lock` when you need flexibility, `scoped_lock` for multi-mutex locking.

Example

```
std::mutex m1, m2;

// lock_guard: simplest
{ std::lock_guard<std::mutex> lk(m1); }

// unique_lock: for condition variables and flexibility
std::unique_lock<std::mutex> lk(m1);
lk.unlock();          // manual unlock
lk.lock();             // manual lock
lk.try_lock();         // non-blocking
auto lk2 = std::move(lk); // movable

// Deferred locking
std::unique_lock<std::mutex> u(m1, std::defer_lock);
u.lock(); // lock later
```

```
// scoped_lock: deadlock-safe multi-mutex (C++17)
{ std::scoped_lock lk(m1, m2); } // locks both atomically

// Old way (deadlock-prone without std::lock):
// std::lock(m1, m2); // lock both, then adopt
// std::lock_guard<std::mutex> g1(m1, std::adopt_lock);
// std::lock_guard<std::mutex> g2(m2, std::adopt_lock);
```

Q10. What is a deadlock and what are the four conditions that cause it? [Mutex] [Pitfall]

Answer

A deadlock is a state where two or more threads wait forever for resources held by each other. The four Coffman conditions that must ALL hold for a deadlock: (1) Mutual Exclusion: resource is non-shareable. (2) Hold and Wait: thread holds a resource while waiting for another. (3) No Preemption: resources cannot be forcibly taken. (4) Circular Wait: thread A waits for resource held by B, B waits for resource held by A. Breaking any one condition prevents deadlock.

Example

```
// Classic deadlock: two threads, two mutexes
std::mutex m1, m2;

// Thread A: locks m1 then tries m2
void threadA() {
    std::lock_guard lk1(m1);
    std::this_thread::sleep_for(1ms); // T2 locks m2
    std::lock_guard lk2(m2); // DEADLOCK: m2 held by B
}

// Thread B: locks m2 then tries m1
void threadB() {
    std::lock_guard lk2(m2);
    std::lock_guard lk1(m1); // DEADLOCK: m1 held by A
}

// FIX 1: consistent lock ordering
void fixedA() { std::lock_guard l1(m1); std::lock_guard l2(m2); }
void fixedB() { std::lock_guard l1(m1); std::lock_guard l2(m2); }

// FIX 2: scoped_lock (C++17)
void fixedSafe() { std::scoped_lock lk(m1, m2); }
```

Q11. What is livelock and starvation, and how do they differ from deadlock? [Mutex] [Pitfall]

Answer

Deadlock: threads are permanently blocked, making no progress. Livelock: threads are NOT blocked but keep responding to each other without making real progress (like two people in a hallway stepping the same way). Starvation: a thread repeatedly loses resource contention and never gets CPU time — it can make progress in theory but never does in practice. Livelock and starvation are harder to detect than deadlock.

Example

```
// Livelock example: two polite threads
std::atomic<bool> m1Held{false}, m2Held{false};

void threadA() {
    while (true) {
        m1Held = true;
        std::this_thread::yield();
    }
}
```

```

        if (m2Held) { m1Held = false; continue; } // back off
        // do work...
        m1Held = false; break;
    }
}
// Both threads keep backing off forever -> livelock

// Fix livelock: randomized backoff
std::this_thread::sleep_for(
    std::chrono::milliseconds(rand() % 10));

// Starvation: low-priority thread never scheduled
// Fix: use fair locks, priority inheritance, or FIFO ordering

```

Q12. What is the double-checked locking pattern and why is it broken without atomics? [Mutex] [Advanced]

Answer

Double-checked locking tries to avoid lock overhead on the common path by checking a flag before acquiring the lock, then re-checking inside the lock. Without memory ordering guarantees, the compiler and CPU can reorder operations — a thread may see the flag set (singleton non-null) but the object's constructor has not been completed yet. In C++11, use `std::call_once` or `atomic` with `acquire/release` for correct DCL.

Example

```

// BROKEN: without proper memory ordering
Singleton* instance = nullptr;
std::mutex m;

Singleton* getBroken() {
    if (!instance) { // First check (no lock)
        std::lock_guard lk(m);
        if (!instance) // Second check (with lock)
            instance = new Singleton(); // REORDERING possible!
    }
    return instance;
}

// CORRECT: atomic with proper ordering
std::atomic<Singleton*> atomicInst{nullptr};

Singleton* getCorrect() {
    auto* p = atomicInst.load(std::memory_order_acquire);
    if (!p) {
        std::lock_guard lk(m);
        p = atomicInst.load(std::memory_order_relaxed);
        if (!p) {
            p = new Singleton();
            atomicInst.store(p, std::memory_order_release);
        }
    }
    return p;
}

// BEST: std::call_once
std::once_flag flag;
Singleton* s = nullptr;
Singleton* get() {
    std::call_once(flag, []{ s = new Singleton(); });
    return s;
}

```

Q13. What is `std::call_once` and `std::once_flag`? [Mutex]**Answer**

`std::call_once` ensures a callable is executed exactly once across all threads, even if multiple threads call it concurrently. `std::once_flag` tracks whether execution has occurred. Subsequent calls to `call_once` with the same flag are no-ops. If the callable throws, the flag is not set and another thread can try. This is the standard way to implement lazy initialization and singletons.

Example

```
#include <mutex>

std::once_flag initFlag;
ExpensiveResource* resource = nullptr;

void initResource() {
    resource = new ExpensiveResource();
    std::cout << "Initialized once\n";
}

ExpensiveResource& getResource() {
    std::call_once(initFlag, initResource);
    return *resource;
}

// Thread-safe singleton using call_once
class Singleton {
    static std::once_flag flag_;
    static std::unique_ptr<Singleton> inst_;
    Singleton() = default;
public:
    static Singleton& get() {
        std::call_once(flag_, []{ inst_.reset(new Singleton()); });
        return *inst_;
    }
};

// Even simpler: C++11 Meyers Singleton (also thread-safe)
Singleton& get() {
    static Singleton instance; // guaranteed thread-safe init
    return instance;
}
```

SECTION 3 — Condition Variables & Synchronization**Q14. What is a condition variable and how does `wait()` work? [Condition Variable]****Answer**

A condition variable allows threads to block (sleep) until a condition becomes true, rather than spinning. `wait()` atomically: (1) releases the mutex, (2) suspends the thread. When notified, it: (1) re-acquires the mutex, (2) checks the predicate. Always use the predicate form of `wait()` to handle spurious wakeups — the OS may wake threads even without `notify_one/all` being called.

Example

```
#include <condition_variable>
```

```

std::mutex m;
std::condition_variable cv;
bool ready = false;

// Consumer thread
void consumer() {
    std::unique_lock<std::mutex> lk(m);
    // Predicate form: loop until ready is true
    cv.wait(lk, []{ return ready; }); // handles spurious wakeups
    // OR equivalently:
    // while (!ready) cv.wait(lk);
    process();
}

// Producer thread
void producer() {
    {
        std::lock_guard<std::mutex> lk(m);
        ready = true; // set condition BEFORE notifying
    } // release lock BEFORE notifying (best practice)
    cv.notify_one(); // wake one waiting thread
    // cv.notify_all(); // wake all waiting threads
}

```

Q15. What are spurious wakeups and how do you handle them? [Condition Variable] [Pitfall]

Answer

A spurious wakeup is when a thread waiting on a condition variable wakes up without `notify_one/all` being called. This is allowed by POSIX and happens on real hardware. Without a predicate, your thread proceeds with the condition still false — processing garbage. Always wrap `wait()` with a predicate or a while loop that re-checks the condition after every wakeup.

Example

```

// WRONG: no predicate -- spurious wakeup causes bug
std::unique_lock<std::mutex> lk(m);
cv.wait(lk); // may wake spuriously!
process(); // condition might still be false!

// CORRECT: predicate form (internally loops)
cv.wait(lk, [&]{ return !queue.empty(); });
process(queue.front()); // safe: condition is definitely true

// EQUIVALENT manual loop:
while (queue.empty()) {
    cv.wait(lk); // release+sleep; reacquire on wake
    // re-check: was it spurious or real?
}
process(queue.front());

// With timeout:
bool notified = cv.wait_for(lk,
    std::chrono::seconds(5),
    [&]{ return !queue.empty(); });
if (!notified) { /* timed out */ }

```

Q16. What is the difference between `notify_one()` and `notify_all()`? [Condition Variable]

Answer

`notify_one()` wakes exactly one waiting thread (which one is unspecified). `notify_all()` wakes all waiting threads, each of which re-acquires the mutex and re-checks its predicate — only threads whose predicate is true will proceed. Use `notify_one` when only one thread can act (single resource). Use `notify_all` when multiple threads might be able to proceed, or when the condition change affects all waiters.

Example

```
std::condition_variable cv;
std::mutex m;
bool dataReady = false;

// Producer with notify_one: one consumer processes one item
void produceOne() {
    { std::lock_guard lk(m); dataReady = true; }
    cv.notify_one(); // wake exactly one consumer
}

// All consumers shutdown: use notify_all
bool shutdown = false;
void shutdownAll() {
    { std::lock_guard lk(m); shutdown = true; }
    cv.notify_all(); // wake ALL consumers so they can exit
}

// Consumer checks both conditions
void consumer() {
    std::unique_lock lk(m);
    cv.wait(lk, [&]{ return dataReady || shutdown; });
    if (shutdown) return;
    processData();
    dataReady = false;
}
```

Q17. Explain `std::promise`, `std::future`, and `std::shared_future`. [\[Future\]](#) [\[Promise\]](#)

Answer

`promise<T>` is the write end: `set_value()` or `set_exception()` is called from the producer thread. `future<T>` is the read end: `get()` blocks until the value is available, then returns it (or rethrows the exception). `get()` can only be called once. `shared_future<T>` can be copied and `get()` called multiple times from multiple threads — used when multiple consumers need the same result.

Example

```
// promise/future: one producer, one consumer
std::promise<int> prom;
std::future<int> fut = prom.get_future();

std::thread producer([&prom]{
    try {
        int result = compute();
        prom.set_value(result);
    } catch (...) {
        prom.set_exception(std::current_exception());
    }
});

int val = fut.get(); // blocks; rethrows if exception
producer.join();

// shared_future: multiple consumers
std::promise<Config> cp;
std::shared_future<Config> sf = cp.get_future().share();

// Multiple threads can all call sf.get()
```

```
std::thread t1([sf]{ use(sf.get()); });
std::thread t2([sf]{ use(sf.get()); });
cp.set_value(loadConfig());
t1.join(); t2.join();
```

Q18. What is `std::async` and what are its launch policies? [Future] [Async]

Answer

`std::async` runs a callable asynchronously and returns a future. `launch::async` guarantees a new thread is created. `launch::deferred` runs the callable lazily in the calling thread when `get()` or `wait()` is called. The default (`launch::async | launch::deferred`) lets the implementation choose — this can result in deferred execution (no parallelism!). Always specify `launch::async` explicitly when you need a new thread.

Example

```
#include <future>

// Guaranteed async: new thread
auto f1 = std::async(std::launch::async,
    []{ return compute(); });

// Deferred: runs in caller thread at get() time
auto f2 = std::async(std::launch::deferred,
    []{ return compute(); });
// No thread created yet!
f2.get(); // compute() runs HERE, in this thread

// DANGER: default policy might defer
auto f3 = std::async([]{ return compute(); }); // might defer!

// Exception propagation
auto f4 = std::async(std::launch::async,
    []{ throw std::runtime_error("oops"); return 0; });
try {
    f4.get(); // rethrows the exception
} catch (const std::runtime_error& e) {
    std::cout << e.what();
}

// IMPORTANT: future destructor blocks if async was used!
// std::async(launch::async, fn); // destructor blocks here
```

Q19. What are `std::counting_semaphore` and `std::binary_semaphore` (C++20)? [Semaphore] [C++20]

Answer

A semaphore is a signaling primitive that maintains a counter. `acquire()` decrements it (blocks if 0). `release()` increments it and wakes a blocked thread. `counting_semaphore<max>` has a maximum count. `binary_semaphore` is a `counting_semaphore<1>` — effectively a signaling mutex. Unlike mutex, a semaphore can be released by a DIFFERENT thread than the one that acquired it, making it ideal for producer-consumer signaling.

Example

```
#include <semaphore>

// Binary semaphore as a signal
std::binary_semaphore signal{0}; // starts at 0 (blocked)

std::thread producer([&]{
```

```

    doWork();
    signal.release(); // signal consumer (from different thread!)
});

signal.acquire(); // wait for producer
consumeResult();

// Counting semaphore: limit concurrent access
std::counting_semaphore<10> pool{10}; // 10 permits

void useResource() {
    pool.acquire();      // decrement; block if 0
    doWork();
    pool.release();      // increment; wake a waiter
}

// try_acquire: non-blocking
if (pool.try_acquire()) {
    doWork();
    pool.release();
}

// try_acquire_for: timed
if (pool.try_acquire_for(std::chrono::ms(100))) { }
```

Q20. What are `std::latch` and `std::barrier` (C++20)? [Latch] [Barrier] [C++20]

Answer

`std::latch` is a single-use countdown synchronization point. Threads call `count_down()` to decrement the counter; `wait()` blocks until count reaches zero. Not resettable. `std::barrier` is a reusable synchronization point where all participating threads must arrive before any can proceed. It optionally calls a completion function between phases. Ideal for fork-join parallel algorithms.

Example

```

#include <latch>
#include <barrier>

// std::latch: wait for N tasks to complete
std::latch done{5}; // 5 workers

for (int i = 0; i < 5; ++i)
    std::thread([&done, i]{
        doWork(i);
        done.count_down(); // signal completion
    }).detach();

done.wait(); // wait for all 5

// std::barrier: synchronize N threads across phases
std::barrier sync{4}; // 4 threads

auto worker = [&sync](int id) {
    for (int phase = 0; phase < 3; ++phase) {
        doPhaseWork(id, phase);
        sync.arrive_and_wait(); // all must reach here
        // Next phase starts only when all arrive
    }
};

// Barrier with completion function
std::barrier syncEx{4, []() noexcept {
    std::cout << "Phase complete!\n"; // runs once per phase
}}
```

```
}};
```

SECTION 4 — Atomic Operations & Memory Model

Q21. What is `std::atomic` and what operations does it support? [Atomic]

Answer

`std::atomic<T>` provides lock-free atomic operations on a variable — load, store, exchange, `compare_exchange`, and `fetch_add/sub/and/or/xor`. All operations are indivisible (no torn reads/writes). Specializations for `bool`, integral types, and pointers provide additional arithmetic. Atomic operations have `memory_order` parameters controlling visibility and ordering.

Example

```
#include <atomic>

std::atomic<int> counter{0};

// Basic operations
counter.store(5);           // atomic write
int val = counter.load();   // atomic read
int old = counter.exchange(10); // atomic swap
counter.fetch_add(1);        // atomic increment
counter.fetch_sub(1);        // atomic decrement
counter++; // same as fetch_add(1)

// compare_exchange: CAS (Compare And Swap)
int expected = 10;
bool swapped = counter.compare_exchange_strong(
    expected, 20); // if counter==expected, store 20
// expected updated to actual value if swap failed

// Atomic flag (guaranteed lock-free)
std::atomic_flag flag = ATOMIC_FLAG_INIT;
bool wasSet = flag.test_and_set(); // set flag, return old
flag.clear(); // clear flag

// Check lock-free
std::cout << counter.is_lock_free(); // 1 on most platforms
```

Q22. What are memory orders and when should each be used? [Atomic] [Memory Order]

Answer

`memory_order_relaxed`: no ordering guarantees, only atomicity — use for counters where order does not matter. `memory_order_release`: all preceding writes are visible before this store. `memory_order_acquire`: all subsequent reads see writes that preceded the release. Together they form release-acquire pairs. `memory_order_acq_rel`: both acquire and release (for read-modify-write). `memory_order_seq_cst`: total sequential order (default, strongest, most expensive). Always use `seq_cst` unless you have proven the weaker order is correct.

Example

```
std::atomic<int> data{0};
std::atomic<bool> ready{false};
```

```
// Producer: write data, then signal with release
void producer() {
    data.store(42, std::memory_order_relaxed);
    ready.store(true, std::memory_order_release);
    // All preceding writes visible before release store
}

// Consumer: acquire ensures data is visible
void consumer() {
    while (!ready.load(std::memory_order_acquire));
    // All writes before release are visible here
    assert(data.load(std::memory_order_relaxed) == 42);
}

// Relaxed: counter where order doesn't matter
std::atomic<int> hits{0};
hits.fetch_add(1, std::memory_order_relaxed); // just count

// Sequential consistency (default)
std::atomic<int> x{0};
x.store(1); // seq_cst by default -- most overhead
```

Q23. What is a data race and how does `std::atomic` prevent it? [\[Atomic\]](#) [\[Pitfall\]](#)

Answer

A data race is undefined behavior: two threads concurrently access the same memory location, at least one access is a write, and there is no synchronization. The compiler and CPU can assume no data races and make optimizations that break racy code in unexpected ways. `std::atomic` makes each operation indivisible and adds necessary memory barriers, eliminating data races on the atomic variable itself.

Example

```
// DATA RACE: undefined behavior
int counter = 0;
std::thread t1([&]{ counter++; }); // read-modify-write
std::thread t2([&]{ counter++; }); // concurrent -- UB!
t1.join(); t2.join();
// counter might be 1 instead of 2!

// NO RACE: atomic
std::atomic<int> safeCounter{0};
std::thread t3([&]{ safeCounter++; }); // atomic++
std::thread t4([&]{ safeCounter++; }); // atomic++
t3.join(); t4.join();
std::cout << safeCounter; // always 2

// Detect with ThreadSanitizer (TSan)
// Compile: g++ -fsanitize=thread -g prog.cpp
// TSan catches data races at runtime

// Atomic does NOT make compound operations atomic:
if (safeCounter > 0) safeCounter--; // NOT atomic! race!
// Fix: compare_exchange in a loop
```

Q24. What is `compare_exchange_strong` vs `compare_exchange_weak`? [\[Atomic\]](#)

Answer

Both perform a CAS: if `*this == expected`, store desired and return true; else set `expected = *this` and return false. `compare_exchange_weak` may fail spuriously (return false even when `*this == expected`) on certain architectures — valid in loops where a retry is cheap. `compare_exchange_strong` never fails spuriously — use it when you cannot retry (e.g., in if statements). Both are essential for implementing lock-free algorithms.

Example

```
std::atomic<int> val{0};

// compare_exchange_strong: guaranteed no spurious failure
int expected = 0;
if (val.compare_exchange_strong(expected, 1)) {
    std::cout << "Swapped: 0 -> 1\n";
} else {
    std::cout << "Was " << expected << ", not 0\n";
}

// compare_exchange_weak: use in loops (may be faster on ARM)
int exp = 0;
while (!val.compare_exchange_weak(exp, 1)) {
    exp = 0; // reset expected and retry
}

// Lock-free max update
void atomicMax(std::atomic<int>& maxVal, int newVal) {
    int cur = maxVal.load(std::memory_order_relaxed);
    while (newVal > cur &&
        !maxVal.compare_exchange_weak(cur, newVal,
            std::memory_order_relaxed));
}
```

Q25. What is the happens-before relationship in the C++ memory model? [Memory Model]

[Advanced]

Answer

The C++ memory model defines "happens-before" as a partial order guaranteeing visibility. If A happens-before B, then all memory writes by A are visible to B. Sequenced-before: within a single thread, operations are ordered by program order. Synchronizes-with: a release store synchronizes-with an acquire load of the same atomic. Happens-before = transitive closure of sequenced-before and synchronizes-with. This is the formal basis for all correct concurrent C++ code.

Example

```
// happens-before chain:
// (1) data=42 sequenced-before (2) flag.store(release)
// (2) flag.store(release) synchronizes-with (3) flag.load(acquire)
// (3) flag.load(acquire) sequenced-before (4) use(data)
// Therefore: (1) happens-before (4)

std::atomic<bool> flag{false};
int data = 0;

void producer() {
    data = 42; // (1)
    flag.store(true, std::memory_order_release); // (2)
}

void consumer() {
    while (!flag.load(std::memory_order_acquire)); // (3)
    assert(data == 42); // (4) safe!
}

// Without the release-acquire pair:
// data=42 does NOT necessarily happen-before use(data)
```

```
// -> data might still appear as 0 to consumer (UB)
```

Q26. What is the ABA problem and how do you solve it? [Atomic] [Advanced]

Answer

The ABA problem: thread 1 reads value A, thread 2 changes A->B->A, thread 1 CAS succeeds (sees A) but the context has changed. Common in lock-free stacks: the "same" pointer value may point to a different object (recycled memory). Solutions: tagged pointers (version counter in high bits), hazard pointers, epoch-based reclamation, or avoiding recycled addresses.

Example

```
// ABA in a lock-free stack:
// Stack: A -> B -> C
// Thread 1: reads head=A, next=B
// Thread 2: pops A, pops B, pushes A (recycled!)
// Stack: A -> C
// Thread 1: CAS(head, A, B) SUCCEEDS -- but B is freed!

// Fix: tagged pointer (version counter)
struct TaggedPtr {
    Node* ptr;
    uintptr_t tag; // increment on every CAS
};

// Each CAS bumps the tag -- ABA impossible
// (same pointer + different tag = different value)
std::atomic<TaggedPtr> head;

void push(Node* n) {
    TaggedPtr old = head.load();
    TaggedPtr next;
    do {
        n->next = old.ptr;
        next = {n, old.tag + 1}; // bump tag
    } while (!head.compare_exchange_weak(old, next));
}
```

SECTION 5 — Thread Pool Design & Implementation

Q27. What is a thread pool and why is it preferred over creating threads on demand? [Thread Pool]

Answer

A thread pool pre-creates a fixed number of worker threads that wait for tasks. Thread creation/destruction is expensive (system call, stack allocation ~1MB per thread, TLB flushes). A pool amortizes this cost — tasks are submitted to a queue and picked up by idle workers. Benefits: bounded resource usage, lower latency (no thread creation per task), predictable load, and easier lifetime management.

Example

```
// Why thread creation is expensive:
// - pthread_create: ~15-50 microseconds
// - Default stack: 1-8 MB allocation
// - TLB and cache effects
```

```
// Without pool: one thread per task
for (auto& task : tasks) {
    std::thread t(task); // 50us per task!
    t.detach(); // no RAII -- dangerous
}

// With pool: submit tasks to shared queue
ThreadPool pool(std::thread::hardware_concurrency());

for (auto& task : tasks) {
    pool.submit(task); // microseconds: just queue push
}
// Workers process tasks concurrently

// Pool also provides:
// - Backpressure (bounded queue)
// - Controlled shutdown
// - Future-based result retrieval
```

Q28. Implement a basic thread pool with a task queue and futures. [Thread Pool]

▶ Answer

A production-quality thread pool uses: a lock-protected task queue, condition variable for worker notification, shutdown flag for clean termination, and packaged_task/promise to return results. Workers loop: wait for task, execute it, wait again. submit() wraps the callable in a packaged_task to enable future-based result retrieval.

Example

```
class ThreadPool {
    std::vector<std::thread> workers_;
    std::queue<std::function<void()>> tasks_;
    std::mutex m_;
    std::condition_variable cv_;
    bool stop_ = false;
public:
    explicit ThreadPool(size_t n) {
        for (size_t i = 0; i < n; ++i)
            workers_.emplace_back([this] {
                while (true) {
                    std::function<void()> task;
                    {
                        std::unique_lock lk(m_);
                        cv_.wait(lk, [this]{
                            return stop_ || !tasks_.empty();
                        });
                        if (stop_ && tasks_.empty()) return;
                        task = std::move(tasks_.front());
                        tasks_.pop();
                    }
                    task(); // execute outside lock
                }
            });
    }

    template<typename F, typename... Args>
    auto submit(F&& f, Args&&... args) {
        using Ret = std::invoke_result_t<F, Args...>;
        auto task = std::make_shared<std::packaged_task<Ret()>>(
            std::bind(std::forward<F>(f),
                      std::forward<Args>(args)...));
        std::future<Ret> fut = task->get_future();
        {
            std::lock_guard lk(m_);
```

```

        if (stop_) throw std::runtime_error("stopped");
        tasks_.emplace([task]{ (*task)(); });
    }
    cv_.notify_one();
    return fut;
}

~ThreadPool() {
    { std::lock_guard lk(m_); stop_ = true; }
    cv_.notify_all(); // wake all workers
    for (auto& w : workers_) w.join();
}
};

// Usage
ThreadPool pool(4);
auto f = pool.submit([](int x){ return x*x; }, 7);
std::cout << f.get(); // 49

```

Q29. What is work stealing and how does it improve thread pool performance? [Thread Pool] [Advanced]

Answer

In a basic pool, all threads share one queue (contention). Work stealing gives each thread its own local deque. When a thread runs out of local work, it "steals" from the back of another thread's deque. The owner pushes/pops from the front (LIFO — cache warm data). Stealers take from the back (FIFO — less contention). This improves cache locality and load balancing. Used by Intel TBB, Java ForkJoinPool, Cilk.

Example

```

// Work stealing design:

// Each worker has its own deque
thread_local std::deque<Task> localQueue;

// Worker loop
void workerLoop(int id) {
    while (!done) {
        Task t;
        if (!localQueue.empty()) {
            t = localQueue.front(); // LIFO from own queue
            localQueue.pop_front();
        } else {
            // Try to steal from another thread
            t = stealFrom(randomOtherWorker());
        }
        if (t) execute(t);
        else std::this_thread::yield();
    }
}

// Submit: push to submitting thread's local queue
void submit(Task t) {
    localQueue.push_front(t); // back of deque
    // If submitting from outside pool, use global queue
}

// Real implementations: Intel TBB, Taskflow, std::execution
#include <tbb/task_group.h>
tbb::task_group tg;
tg.run([]{ doWork(); });
tg.wait();

```

Q30. How do you implement graceful shutdown of a thread pool? [Thread Pool]**Answer**

Graceful shutdown: (1) Drain mode — stop accepting new tasks, finish all queued tasks, then stop workers. (2) Immediate stop — workers finish current task and exit, remaining queued tasks are discarded. The shutdown is signalled via a flag + condition variable. Workers check the flag when the queue is empty. Join all workers in the destructor.

Example

```
class ThreadPool {
    // ... members from Q28 ...

    // Drain: finish remaining tasks then stop
    void shutdown(bool drain = true) {
        if (drain) {
            // Wait until queue is empty
            std::unique_lock lk(m_);
            drainCv_.wait(lk, [this]{ return tasks_.empty(); });
        }
        { std::lock_guard lk(m_); stop_ = true; }
        cv_.notify_all();
        for (auto& w : workers_)
            if (w.joinable()) w.join();
    }

    // Worker signals when queue drains
    // (inside worker loop after pop):
    // if (tasks_.empty()) drainCv_.notify_all();
};

// Usage
ThreadPool pool(4);
for (int i = 0; i < 100; ++i)
    pool.submit([i]{ processItem(i); });

pool.shutdown(true); // wait for all 100 tasks
// pool.shutdown(false); // stop immediately
```

Q31. What is a priority task queue in a thread pool and how do you implement one? [Thread Pool] [Advanced]**Answer**

A priority queue ensures high-priority tasks run before low-priority ones. Replace the `std::queue` with a `std::priority_queue` or sorted container. The task wrapper includes a priority value. Workers always pick the highest priority task. Danger: starvation of low-priority tasks if high-priority tasks keep arriving — use aging (bump priority over time).

Example

```
enum class Priority { Low=0, Normal=1, High=2, Critical=3 };

struct PriorityTask {
    int priority;
    std::function<void()> fn;
    bool operator<(const PriorityTask& o) const {
        return priority < o.priority; // max-heap
    }
};

class PriorityPool {
    std::priority_queue<PriorityTask> tasks_;
```

```

    std::mutex m_;
    std::condition_variable cv_;
    bool stop_ = false;
public:
    void submit(std::function<void()> fn,
                Priority p = Priority::Normal) {
        {
            std::lock_guard lk(m_);
            tasks_.push({static_cast<int>(p), std::move(fn)});
        }
        cv_.notify_one();
    }
    // worker loop: always picks highest priority task
};

PriorityPool pool;
pool.submit([]{ background(); }, Priority::Low);
pool.submit([]{ userRequest(); }, Priority::High);
// userRequest() runs first

```

SECTION 6 — Thread-Safe Containers

Q32. Why are STL containers NOT thread-safe and what are the rules? [\[Thread-Safe Container\]](#)

Answer

STL containers have no internal synchronization. The rules: (1) Multiple concurrent reads are safe (const operations). (2) Any concurrent write (even with other reads) is a data race — undefined behavior. (3) Even "const" operations like `operator[]` on `unordered_map` are NOT safe if the map rehashes internally. You must provide external synchronization for any shared mutable container.

Example

```

std::vector<int> v = {1,2,3};

// SAFE: concurrent reads (no writes)
std::thread t1([&]{ std::cout << v[0]; });
std::thread t2([&]{ std::cout << v[1]; });

// DATA RACE: concurrent read+write
std::thread t3([&]{ v.push_back(4); }); // write
std::thread t4([&]{ std::cout << v[0]; }); // read -- UB!

// FIX: external mutex
std::mutex m;
std::thread t5([&]{
    std::lock_guard lk(m);
    v.push_back(4);
});
std::thread t6([&]{
    std::lock_guard lk(m);
    std::cout << v[0];
});

// Note: Java's Vector/Hashtable have internal locks
// C++ philosophy: you pay only for what you use

```

Q33. Implement a thread-safe queue (producer-consumer). [Thread-Safe Container]**Answer**

A thread-safe queue wraps `std::queue` with a mutex and condition variable. `push()` adds items and notifies. `pop()` waits if empty. `try_pop()` is non-blocking. A shutdown mechanism lets consumers exit cleanly. This is the foundation of all producer-consumer systems and thread pools.

Example

```
template<typename T>
class ThreadSafeQueue {
    std::queue<T> q_;
    mutable std::mutex m_;
    std::condition_variable cv_;
    bool closed_ = false;
public:
    void push(T val) {
        { std::lock_guard lk(m_); q_.push(std::move(val)); }
        cv_.notify_one();
    }

    // Blocking pop: waits until item or closed
    bool pop(T& out) {
        std::unique_lock lk(m_);
        cv_.wait(lk, [this]{ return !q_.empty() || closed_; });
        if (q_.empty()) return false; // closed
        out = std::move(q_.front());
        q_.pop();
        return true;
    }

    // Non-blocking
    bool try_pop(T& out) {
        std::lock_guard lk(m_);
        if (q_.empty()) return false;
        out = std::move(q_.front());
        q_.pop();
        return true;
    }

    void close() {
        { std::lock_guard lk(m_); closed_ = true; }
        cv_.notify_all();
    }

    size_t size() const {
        std::lock_guard lk(m_); return q_.size();
    }
};
```

Q34. Implement a thread-safe stack. [Thread-Safe Container]**Answer**

A thread-safe stack needs to make `top()` + `pop()` atomic — otherwise a race exists between checking empty and popping. The solution: combine them into a single `pop()` that returns an `optional<T>` or a `shared_ptr<T>`. This eliminates the TOCTOU (Time Of Check To Time Of Use) race.

Example

```
template<typename T>
class ThreadSafeStack {
    std::stack<T> s_;
    mutable std::mutex m_;
    std::condition_variable cv_;
public:
```

```

void push(T val) {
    { std::lock_guard lk(m_); s_.push(std::move(val)); }
    cv_.notify_one();
}

// Blocking pop (waits if empty)
T pop() {
    std::unique_lock lk(m_);
    cv_.wait(lk, [this]{ return !s_.empty(); });
    T val = std::move(s_.top());
    s_.pop();
    return val; // NRVO
}

// Non-blocking pop
std::optional<T> try_pop() {
    std::lock_guard lk(m_);
    if (s_.empty()) return std::nullopt;
    T val = std::move(s_.top());
    s_.pop();
    return val;
}

bool empty() const {
    std::lock_guard lk(m_); return s_.empty();
}
};

// WHY combine top+pop?
// std::stack<int> s; // not thread-safe
// if (!s.empty())    // Thread 2 pops here!
//     s.top();        // UB: stack is now empty

```

Q35. Implement a thread-safe hash map with fine-grained locking. [Thread-Safe Container]

Answer

A single mutex for the whole map is simple but creates a bottleneck. Fine-grained locking divides the map into N buckets, each with its own mutex. Threads on different buckets can proceed in parallel. The number of bucket-mutexes (commonly a prime number) should balance overhead and parallelism. Uses `shared_mutex` for read-heavy workloads.

Example

```

template<typename K, typename V, size_t N = 16>
class ShardedMap {
    struct Shard {
        std::unordered_map<K,V> map;
        mutable std::shared_mutex mtx;
    };
    std::array<Shard, N> shards_;

    Shard& getShard(const K& key) {
        return shards_[std::hash<K>{}(key) % N];
    }
    const Shard& getShard(const K& key) const {
        return shards_[std::hash<K>{}(key) % N];
    }
public:
    void put(const K& k, V v) {
        auto& shard = getShard(k);
        std::unique_lock lk(shard.mtx); // exclusive write
        shard.map[k] = std::move(v);
    }
}

```

```

std::optional<V> get(const K& k) const {
    auto& shard = getShard(k);
    std::shared_lock lk(shard.mtx); // shared read
    auto it = shard.map.find(k);
    if (it == shard.map.end()) return std::nullopt;
    return it->second;
}

bool erase(const K& k) {
    auto& shard = getShard(k);
    std::unique_lock lk(shard.mtx);
    return shard.map.erase(k) > 0;
}
};

```

Q36. What is a lock-free single-producer single-consumer (SPSC) queue? [Thread-Safe Container] [Advanced]

▶ Answer

An SPSC queue has exactly one producer and one consumer — no contention between them. A ring buffer with a head (consumer) and tail (producer) index, both atomic, implements this without any mutex. The producer writes to tail and advances; the consumer reads from head and advances. With acquire/release ordering on the indices, no other synchronization is needed. Extremely fast for trading systems and audio processing.

Example

```

template<typename T, size_t Cap>
class SPSCQueue {
    std::array<T, Cap> buf_;
    alignas(64) std::atomic<size_t> head_{0}; // consumer
    alignas(64) std::atomic<size_t> tail_{0}; // producer
    // Separate cache lines to avoid false sharing!
public:
    bool push(T val) { // producer only
        size_t t = tail_.load(std::memory_order_relaxed);
        size_t next = (t + 1) % Cap;
        if (next == head_.load(std::memory_order_acquire))
            return false; // full
        buf_[t] = std::move(val);
        tail_.store(next, std::memory_order_release);
        return true;
    }

    bool pop(T& out) { // consumer only
        size_t h = head_.load(std::memory_order_relaxed);
        if (h == tail_.load(std::memory_order_acquire))
            return false; // empty
        out = std::move(buf_[h]);
        head_.store((h+1)%Cap, std::memory_order_release);
        return true;
    }
};

```

Q37. What is a concurrent bounded buffer (producer-consumer with capacity limit)? [Thread-Safe Container]

▶ Answer

A bounded buffer limits how many items can queue up — providing backpressure to producers when the buffer is full. Uses two semaphores (or two condition variables): one counting empty slots (producer waits when 0),

one counting filled slots (consumer waits when 0). This is the classic dining philosophers / producer-consumer synchronization.

Example

```
template<typename T>
class BoundedBuffer {
    std::vector<T> buf_;
    size_t head_=0, tail_=0, count_=0;
    std::mutex m_;
    std::condition_variable notFull_, notEmpty_;
    const size_t cap_;
public:
    explicit BoundedBuffer(size_t cap)
        : buf_(cap), cap_(cap) {}

    void produce(T val) {
        std::unique_lock lk(m_);
        notFull_.wait(lk, [this]{ return count_ < cap_; });
        buf_[tail_] = std::move(val);
        tail_ = (tail_+1) % cap_;
        ++count_;
        notEmpty_.notify_one();
    }

    T consume() {
        std::unique_lock lk(m_);
        notEmpty_.wait(lk, [this]{ return count_ > 0; });
        T val = std::move(buf_[head_]);
        head_ = (head_+1) % cap_;
        --count_;
        notFull_.notify_one();
        return val;
    }
};
```

Q38. How do you implement a thread-safe LRU cache? [Thread-Safe Container]

Answer

An LRU cache combines a doubly-linked list (order by recency) with a hash map ($O(1)$ lookup). Thread-safe version wraps the whole structure with a `shared_mutex` — readers can hold `shared_lock` concurrently, but a `get()` that moves an item to front needs an exclusive lock. For high concurrency, use a coarse write lock with a `try_shared_lock` for reads.

Example

```
template<typename K, typename V>
class ThreadSafeLRU {
    using List = std::list<std::pair<K,V>>;
    using Iter = typename List::iterator;
    List lru_;
    std::unordered_map<K, Iter> map_;
    size_t cap_;
    mutable std::mutex m_; // or shared_mutex for read opt
public:
    explicit ThreadSafeLRU(size_t cap) : cap_(cap) {}

    std::optional<V> get(const K& k) {
        std::lock_guard lk(m_); // exclusive: moves to front
        auto it = map_.find(k);
        if (it == map_.end()) return std::nullopt;
        lru_.splice(lru_.begin(), lru_, it->second);
        return it->second->second;
    }
};
```

```

void put(const K& k, V v) {
    std::lock_guard lk(m_);
    auto it = map_.find(k);
    if (it != map_.end()) {
        it->second->second = std::move(v);
        lru_.splice(lru_.begin(), lru_, it->second);
        return;
    }
    if (map_.size() >= cap_) {
        map_.erase(lru_.back().first);
        lru_.pop_back();
    }
    lru_.push_front({k, std::move(v)});
    map_[k] = lru_.begin();
}
};

```

SECTION 7 — Concurrency Pitfalls & Anti-Patterns

Q39. What is a race condition (semantic race) vs a data race? [Pitfall]

Answer

A data race is a specific UB: two threads access the same memory concurrently without synchronization, with at least one write. A race condition is a broader semantic issue: the program's behavior depends on the relative timing of events — even with proper synchronization, the logic may be wrong. Example: check-then-act on a file (exists? open it) where another process deletes between check and open. Data race -> UB; race condition -> incorrect behavior but not necessarily UB.

Example

```

// DATA RACE: two threads write without sync -- UB
int x = 0;
std::thread t1([&]{ x = 1; }); // write
std::thread t2([&]{ x = 2; }); // write -- data race!

// RACE CONDITION: sync is correct, logic is not
std::mutex m;
std::unordered_map<int,int> map;

void addIfAbsent(int key, int val) {
    if (map.count(key) == 0) { // check (lock released)
        std::lock_guard lk(m);
        map[key] = val; // act (may overwrite!)
    }
} // Another thread may insert between check and act

// FIX: atomic check-and-insert
void fixedAdd(int key, int val) {
    std::lock_guard lk(m); // hold lock for BOTH
    if (map.count(key) == 0) // check
        map[key] = val; // act
}

```

Q40. What is false sharing and how do you prevent it? [Pitfall] [Memory]

▶ Answer

False sharing occurs when two threads modify different variables that happen to share the same CPU cache line (typically 64 bytes). Even though there is no logical sharing, the cache coherence protocol forces cache line invalidation between CPUs — causing massive performance degradation (10-100x slowdown). Fix: pad variables to separate cache lines using `alignas(64)`.

Example

```
// FALSE SHARING: both in same cache line
struct BadCounters {
    int64_t a; // bytes 0-7
    int64_t b; // bytes 8-15 -- same cache line as a!
};

BadCounters c;
std::thread t1([&]{ for(int i=0;i<1M;i++) c.a++; });
std::thread t2([&]{ for(int i=0;i<1M;i++) c.b++; });
// Each increment invalidates the other's cache line!

// FIX: separate cache lines with padding
struct GoodCounters {
    alignas(64) int64_t a; // cache line 1
    alignas(64) int64_t b; // cache line 2
};

// Or use padding explicitly
struct PaddedCounter {
    int64_t value;
    char pad[64 - sizeof(int64_t)];
};

// Thread pool worker counters -- common pattern
struct alignas(64) WorkerStats {
    std::atomic<uint64_t> tasksProcessed{0};
};
std::vector<WorkerStats> stats(numWorkers);
```

Q41. What are common pitfalls when using condition variables? [Pitfall]**▶ Answer**

(1) Missing predicate: forgetting to loop on the condition causes spurious-wakeup bugs. (2) Notifying before releasing the lock: legal but wakes the waiter who immediately blocks again on the lock — release first. (3) Holding the lock while calling notify: causes unnecessary contention. (4) Lost wakeup: setting the flag but notifying BEFORE the consumer calls `wait()` — the predicate form handles this correctly. (5) Checking the condition without holding the mutex.

Example

```
// PITFALL 1: no predicate -- spurious wakeup bug
cv.wait(lk); // wakes spuriously, processes garbage

// FIX: always use predicate
cv.wait(lk, [&]{ return !queue.empty(); });

// PITFALL 2: notify while holding lock
{
    std::lock_guard lk(m);
    ready = true;
    cv.notify_one(); // waiter wakes but immediately
                    // blocks on the lock we still hold!
}
// FIX: release lock before notifying
{ std::lock_guard lk(m); ready = true; }
cv.notify_one(); // notifier does NOT hold lock
```

```
// PITFALL 3: lost wakeup without predicate
// Producer: ready=true; cv.notify_one();
// Consumer starts AFTER notify -- hangs forever without pred!
// FIX: predicate means wait() checks ready first
cv.wait(lk, [&]{ return ready; }); // sees ready=true immediately
```

Q42. What is the "thread safety of std::shared_ptr" and what pitfalls exist? [Pitfall]

Answer

The reference count in `shared_ptr` IS thread-safe (atomic operations). The OBJECT it points to is NOT thread-safe. The `shared_ptr` instance itself is NOT thread-safe: copying/destroying a `shared_ptr` from multiple threads requires synchronization. Specifically: concurrently reading the same `shared_ptr` is safe; concurrent read+write of the same `shared_ptr` instance is a data race (even though the count is atomic).

Example

```
auto sp = std::make_shared<int>(42);

// SAFE: concurrent reads of same shared_ptr
std::thread t1([sp]{ std::cout << *sp; }); // copy: count++
std::thread t2([sp]{ std::cout << *sp; }); // copy: count++

// UNSAFE: concurrent read+write of same shared_ptr INSTANCE
std::shared_ptr<int> shared = std::make_shared<int>(0);
std::thread t3([&]{ shared = std::make_shared<int>(1); }); // write
std::thread t4([&]{ auto x = shared; }); // read -- DATA RACE!

// FIX: use atomic<shared_ptr> (C++20)
std::atomic<std::shared_ptr<int>> atomicSp =
    std::make_shared<int>(0);
atomicSp.store(std::make_shared<int>(1)); // atomic write
auto val = atomicSp.load(); // atomic read

// The pointed-to object needs its own synchronization
auto sp2 = std::make_shared<int>(0);
std::thread t5([sp2]{ (*sp2)++; }); // race on *sp2!
// use atomic<int> or mutex inside the object
```

Q43. What is priority inversion and how is it solved? [Pitfall]

Answer

Priority inversion: a high-priority thread waits for a resource held by a low-priority thread, which is preempted by medium-priority threads. The high-priority thread is effectively blocked by medium-priority threads. Classic example: Mars Pathfinder 1997. Solutions: (1) Priority inheritance — the low-priority thread temporarily gets the high-priority thread's priority. (2) Priority ceiling — a mutex has a ceiling priority assigned to all lockers. (3) Lock-free algorithms.

Example

```
// Priority inversion scenario:
// High (H) waits for mutex held by Low (L)
// Medium (M) preempts L -> H waits for M to finish
// H effectively has lower priority than M!

// POSIX priority inheritance mutex
pthread_mutexattr_t attr;
pthread_mutexattr_init(&attr);
pthread_mutexattr_setprotocol(&attr, PTHREAD_PRIO_INHERIT);
// Low thread inherits High's priority when High waits
```

```
// In C++: no standard support for priority inheritance
// Use platform-specific APIs or RTOS primitives

// Practical mitigations in C++:
// 1. Minimize lock hold time
// 2. Avoid long operations under lock
// 3. Use lock-free structures for critical paths
// 4. Separate high/low priority work into different queues

// Trading system example: use dedicated thread for
// order execution, never block it on low-priority work
```

Q44. What are common pitfalls with `std::async`? [Pitfall]

▶ Answer

(1) Default launch policy may defer execution — no parallelism. (2) The future returned by `async` blocks in its destructor if it was launched with `async` policy — not fire-and-forget. (3) If you discard the future, the destructor blocks right there. (4) Exceptions from `async` are stored and rethrown at `get()` — silent until then. (5) No control over which thread executes the task.

Example

```
// PITFALL 1: default policy may defer
auto f = std::async([]{ compute(); }); // might not run yet!

// FIX: always specify async
auto f2 = std::async(std::launch::async, []{ compute(); });

// PITFALL 2: discarding future blocks!
std::async(std::launch::async, []{ longTask(); });
// ^ temporary future destroyed here -- BLOCKS until done!

// FIX: store the future
auto fut = std::async(std::launch::async, []{ longTask(); });
// ... do other work ...
fut.get(); // explicitly wait when ready

// PITFALL 3: silent exception
auto f3 = std::async(std::launch::async,
    []{ throw std::runtime_error("error"); });
// No crash yet!
f3.get(); // exception rethrown here

// PITFALL 4: no thread pool -- each async may create a thread
// For many tasks, use a thread pool instead
```

Q45. What is a benign race and why is it never actually safe? [Pitfall]

▶ Answer

A "benign race" is a data race where the programmer believes the wrong value is harmless (e.g., a flag being read slightly stale). However, the C++ standard classifies ALL data races as undefined behavior. The compiler may cache the variable in a register, assuming no concurrent writes, and never re-read from memory. Cache coherence is not guaranteed without explicit memory operations. There are no benign data races in C++.

Example

```
// "Benign" race -- STILL UNDEFINED BEHAVIOR
bool done = false;
```

```
std::thread worker([&]{
    doWork();
    done = true; // write without sync
});

// "Just checking" -- but still a race
while (!done) { // read without sync -- UB!
    std::this_thread::yield();
}
// Compiler may hoist done check out of loop!
// Compiled to: if (!done) while(true) yield();

// FIX: atomic
std::atomic<bool> atomicDone{false};
std::thread worker2([&]{
    doWork();
    atomicDone.store(true, std::memory_order_release);
});
while (!atomicDone.load(std::memory_order_acquire))
    std::this_thread::yield();
```

Q46. What is lock elision and contention and how do you reduce it? [Pitfall]

▶ Answer

Lock contention occurs when multiple threads compete for the same mutex, causing serialization and waking/sleeping overhead. Strategies to reduce: (1) Reduce lock scope (hold for minimum time). (2) Fine-grained locking (per-bucket, per-object). (3) Reader-writer locks for read-heavy workloads. (4) Lock-free data structures. (5) Lock elision (transactional memory, Intel TSX). (6) Thread-local storage to avoid sharing.

Example

```
// HIGH CONTENTION: large critical section
void processAll(std::vector<Item>& items) {
    std::lock_guard lk(m_);
    for (auto& item : items) { // holds lock for entire loop
        expensiveProcess(item);
    }
}

// LOW CONTENTION: minimize lock scope
void processAll2(std::vector<Item>& items) {
    for (auto& item : items) {
        Item copy;
        { std::lock_guard lk(m_); copy = item; } // brief lock
        expensiveProcess(copy); // outside lock
        { std::lock_guard lk(m_); storeResult(copy); }
    }
}

// Thread-local accumulation: no contention
thread_local int localSum = 0;
void addToSum(int val) { localSum += val; } // no lock!

// Merge thread-local results at the end
std::atomic<int> globalSum{0};
void mergeLocal() {
    globalSum.fetch_add(localSum, std::memory_order_relaxed);
}
```

SECTION 8 — Advanced Concurrency Patterns

Q47. What is the Active Object pattern? [Pattern]

▶ Answer

The Active Object pattern decouples method invocation from execution — each "active object" has its own thread and a message queue. Callers submit method requests to the queue (returning futures) and the object's thread executes them serially. This eliminates the need for external locking on the object's state. Common in event-driven systems and UI threads.

Example

```
class ActiveBankAccount {
    double balance_ = 0;
    ThreadSafeQueue<std::function<void()>> q_;
    std::thread worker_;

    void run() {
        std::function<void()> task;
        while (q_.pop(task)) task();
    }
public:
    ActiveBankAccount() : worker_([this]{ run(); }) {}
    ~ActiveBankAccount() { q_.close(); worker_.join(); }

    std::future<void> deposit(double amount) {
        auto prom = std::make_shared<std::promise<void>>();
        auto fut = prom->get_future();
        q_.push([this, amount, prom]{
            balance_ += amount; // runs on worker thread
            prom->set_value();
        });
        return fut;
    }

    std::future<double> balance() {
        auto prom = std::make_shared<std::promise<double>>();
        auto fut = prom->get_future();
        q_.push([this, prom]{ prom->set_value(balance_); });
        return fut;
    }
};
```

Q48. What is the Read-Copy-Update (RCU) pattern? [Pattern]

▶ Answer

RCU allows concurrent reads without any locking while making updates visible atomically. Writers create a new copy of the data, make it visible via an atomic pointer swap, then wait for all existing readers to finish before freeing the old copy. Readers access data inside a critical section (not a mutex — just a flag preventing the old copy from being freed). Used in Linux kernel, Folly, and high-performance databases.

Example

```
// Simplified user-space RCU pattern
struct Config { std::string server; int port; };

class RCUConfig {
    std::atomic<std::shared_ptr<Config>> current_;
public:
    RCUConfig(Config c)
        : current_(std::make_shared<Config>(std::move(c))) {}
};
```

```

// READERS: lock-free, very fast
std::shared_ptr<Config> read() const {
    return current_.load(std::memory_order_acquire);
}

// WRITERS: create new copy, swap atomically
void update(Config newCfg) {
    auto newPtr = std::make_shared<Config>(std::move(newCfg));
    auto old = current_.exchange(newPtr,
        std::memory_order_acq_rel);
    // old shared_ptr destroyed when all readers done
    // (shared_ptr reference count == RCU grace period)
}

};

// Usage
RCUConfig cfg({"db1", 5432});
// Reader: snapshot, no blocking
auto snap = cfg.read(); // lock-free!
connect(snap->server, snap->port);
// Writer: O(1) update
cfg.update({"db2", 5433});

```

Q49. What is a pipeline pattern for parallel processing? [\[Pattern\]](#)

Answer

A pipeline splits processing into stages, each running in a dedicated thread or thread pool, connected by queues. Items flow through stage-by-stage. Throughput = bottleneck stage throughput. Latency = sum of stage latencies. Each stage is independent — can be optimized or scaled independently. Used in media processing, ETL pipelines, and trading order processing.

Example

```

template<typename In, typename Out>
class PipelineStage {
    using Fn = std::function<Out(In)>;
    ThreadSafeQueue<In>* input_;
    ThreadSafeQueue<Out>* output_;
    Fn transform_;
    std::jthread worker_;
public:
    PipelineStage(ThreadSafeQueue<In>& in,
        ThreadSafeQueue<Out>& out,
        Fn fn)
        : input_(&in), output_(&out),
          transform_(std::move(fn)),
          worker_([this](std::stop_token st) {
              In item;
              while (!st.stop_requested() && input_->pop(item))
                  output_->push(transform_(std::move(item)));
          }) {}
};

// Pipeline: read -> decode -> process -> encode -> write
ThreadSafeQueue<RawData>    q1;
ThreadSafeQueue<Frame>      q2;
ThreadSafeQueue<Processed>  q3;

PipelineStage s1(q1, q2, decode);
PipelineStage s2(q2, q3, process);
// Items flow through automatically

```

Q50. What is the reactor pattern and how does it relate to threading? [Pattern]**Answer**

The Reactor pattern uses a single event loop thread (or small pool) that dispatches I/O events to handlers. It avoids the cost of one-thread-per-connection. The thread calls select/epoll/kqueue, gets notified when I/O is ready, and calls the registered handler. All handlers run on the reactor thread — they must NOT block. Blocking work is offloaded to a thread pool. Foundation of Node.js, nginx, Boost.Asio.

Example

```
// Simplified reactor (epoll-based)
class Reactor {
    int epollFd_;
    std::unordered_map<int, std::function<void()>> handlers_;
    ThreadPool& workers_; // for blocking work
public:
    void addHandler(int fd, std::function<void()> handler) {
        handlers_[fd] = std::move(handler);
        epoll_event ev{EPOLLIN, {.fd=fd}};
        epoll_ctl(epollFd_, EPOLL_CTL_ADD, fd, &ev);
    }

    void run() { // single reactor thread
        epoll_event events[64];
        while (running_) {
            int n = epoll_wait(epollFd_, events, 64, -1);
            for (int i = 0; i < n; ++i) {
                auto fd = events[i].data.fd;
                // Call handler -- MUST NOT block!
                handlers_[fd]();
            }
        }
    }

    void dispatchToPool(std::function<void()> f) {
        workers_.submit(std::move(f)); // offload blocking
    }
};
```

Q51. What is a wait-free algorithm vs lock-free vs obstruction-free? [Pattern] [Advanced]**Answer**

Lock-free: system-wide progress guaranteed — at least ONE thread makes progress in a finite number of steps (others may starve). Wait-free: per-thread progress — EVERY thread completes its operation in bounded steps (no starvation). Obstruction-free: a thread makes progress if it eventually runs alone (weakest). Lock-based: no guarantee — a suspended lock-holder blocks all others. Most production "lock-free" code is actually lock-free; true wait-free is very hard to implement.

Example

```
// LOCK-FREE: CAS loop -- at least one thread always wins
void lockFreeIncrement(std::atomic<int>& v) {
    int old = v.load(std::memory_order_relaxed);
    while (!v.compare_exchange_weak(old, old+1,
        std::memory_order_relaxed)) {} // retry on failure
    // One thread wins each CAS; others retry -- NOT wait-free
}

// WAIT-FREE: fetch_add is genuinely wait-free on x86
void waitFreeIncrement(std::atomic<int>& v) {
    v.fetch_add(1, std::memory_order_relaxed);
    // Guaranteed to complete in ONE operation, no retry
}
```

```

}

// Hierarchy (weakest to strongest):
// obstruction-free < lock-free < wait-free
// Sequential < lock-based (no guarantee)

// Practical: fetch_add, exchange are wait-free
// CAS loops are lock-free (not wait-free)
// Complex lock-free structures: lock-free queues, lists

```

Q52. What is a memory pool allocator and how does it help in concurrent code? [Pattern]

Answer

The global heap (malloc/new) is protected by an internal lock — in multi-threaded code, frequent allocations become a contention point. A memory pool pre-allocates a slab and dispenses fixed-size chunks. For thread-local pools there is no synchronization at all. For shared pools, a lock-free free-list (atomic push/pop) eliminates mutex overhead. Essential for high-frequency trading and game engines.

Example

```

// Lock-free free-list memory pool
template<typename T, size_t Cap>
class LockFreePool {
    struct Node { Node* next; alignas(T) char storage[sizeof(T)]; };
    std::array<Node, Cap> storage_;
    std::atomic<Node*> freeList_;
public:
    LockFreePool() {
        for (size_t i = 0; i < Cap-1; ++i)
            storage_[i].next = &storage_[i+1];
        storage_[Cap-1].next = nullptr;
        freeList_.store(&storage_[0]);
    }

    T* allocate() {
        Node* old;
        do { old = freeList_.load(); } while (
            old && !freeList_.compare_exchange_weak(old, old->next));
        return old ? reinterpret_cast<T*>(old->storage) : nullptr;
    }

    void deallocate(T* ptr) {
        auto* node = reinterpret_cast<Node*>(ptr);
        Node* old = freeList_.load();
        do { node->next = old; }
        while (!freeList_.compare_exchange_weak(old, node));
    }
};

```

Q53. What is a strand (serialized executor) and when is it useful? [Pattern]

Answer

A strand is an executor that guarantees serial execution of submitted tasks — no two tasks run concurrently — without using a mutex explicitly. It queues tasks and dispatches them one-at-a-time to an underlying thread pool. This gives the object thread-safety without internal locking: the strand IS the synchronization. Used extensively in Boost.Asio and asynchronous systems.

Example

```

// Strand implementation
class Strand {

```

```

ThreadPool& pool_;
std::queue<std::function<void()>> q_;
std::mutex m_;
bool running_ = false;

void dispatch() {
    while (true) {
        std::function<void()> task;
        {
            std::lock_guard lk(m_);
            if (q_.empty()) { running_ = false; return; }
            task = std::move(q_.front()); q_.pop();
        }
        task(); // run outside lock but serially
    }
}

public:
    explicit Strand(ThreadPool& p) : pool_(p) {}

    void post(std::function<void()> f) {
        bool startDispatch = false;
        { std::lock_guard lk(m_);
          q_.push(std::move(f));
          if (!running_) { running_=true; startDispatch=true; }
        }
        if (startDispatch)
            pool_.submit([this]{ dispatch(); });
    }
};

// Object uses strand instead of mutex
class SafeCounter {
    int count_ = 0; Strand strand_;
public:
    void increment() {
        strand_.post([this]{ ++count_; }); // no mutex needed
    }
};

```

Q54. What are C++20 coroutines and how do they relate to threading? [Pattern] [C++20]

Answer

Coroutines are functions that can be suspended (`co_await`) and resumed without a thread context switch. They enable cooperative multitasking on a thread pool: while a coroutine awaits an I/O event, the thread picks up another coroutine. This gives the concurrency of many threads with the cost of few. Unlike threads, coroutine switches are very cheap (no OS involvement). Requires an executor/scheduler to manage scheduling.

Example

```

#include <coroutine>

// A simple Task coroutine type
struct Task {
    struct promise_type {
        Task get_return_object() { return {}; }
        std::suspend_never initial_suspend() { return {}; }
        std::suspend_never final_suspend() noexcept { return {}; }
        void return_void() {}
        void unhandled_exception() {}
    };
};

// Coroutine: suspends at co_await, resumes when ready
Task fetchAndProcess(Executor& ex) {

```

```

auto data = co_await ex.asyncRead("file.txt");
// While awaiting, thread picks up other coroutines!
auto result = co_await ex.asyncProcess(data);
co_await ex.asyncWrite("out.txt", result);
}

// Key: one thread can run THOUSANDS of coroutines
// vs one thread per request (too expensive)

// Libraries: cppcoro, libunifex, Boost.Asio, CppCoro

```

SECTION 9 — Diagnostics, Testing & Best Practices

Q55. How do you detect data races with ThreadSanitizer (TSan)? [\[Diagnostics\]](#)

▶ Answer

TSan instruments memory accesses and detects data races at runtime. It maintains a "shadow" memory tracking which threads accessed each byte and when. When a race is detected it prints a detailed report: the two conflicting accesses, their threads, and their stack traces. Compile with `-fsanitize=thread -g`. TSan has ~5-10x runtime overhead and 5-10x memory overhead. It detects races that actually occur during the run — not all possible races.

Example

```

// Compile: g++ -fsanitize=thread -g -O1 prog.cpp -o prog
// Run: ./prog

// TSan will catch this:
int shared = 0;

std::thread t1([&]{ shared = 1; }); // write
std::thread t2([&]{ shared = 2; }); // write -- race!
t1.join(); t2.join();

// TSan output:
// WARNING: ThreadSanitizer: data race
// Write of size 4 at 0x... by thread T2:
//   #0 <lambda>() prog.cpp:6
// Previous write of size 4 at 0x... by thread T1:
//   #0 <lambda>() prog.cpp:5

// TSan also detects:
// - Destruction of a locked mutex
// - Thread leaks (joinable thread destroyed)
// - Mutex misuse (unlock from wrong thread)

```

Q56. How do you unit test concurrent code effectively? [\[Testing\]](#)

▶ Answer

Concurrency bugs are timing-dependent and hard to reproduce. Strategies: (1) Use dependency injection to control thread scheduling in tests. (2) Test with stress tests (many iterations, many threads). (3) Use synchronization barriers to force interleaving. (4) Use TSan/Helgrind to detect races. (5) Test with sanitizers

and debug builds. (6) Mock the clock for time-dependent code. (7) Use deterministic concurrency testing tools (CDSChecker, Relacy).

Example

```
// Strategy 1: inject barriers to force interleaving
class ThreadSafeCounter {
    int count_ = 0; std::mutex m_;
    std::function<void()> testHook_; // for testing
public:
    void increment() {
        std::lock_guard lk(m_);
        if (testHook_) testHook_(); // pause for test control
        ++count_;
    }
    void setHook(std::function<void()> h) { testHook_ = h; }
};

// Strategy 2: stress test
void stressTest() {
    for (int run = 0; run < 10000; ++run) {
        ThreadSafeCounter c;
        std::vector<std::thread> ts;
        for (int i = 0; i < 10; ++i)
            ts.emplace_back([&c]{ c.increment(); });
        for (auto& t : ts) t.join();
        assert(c.value() == 10);
    }
}
```

Q57. What are the key rules for writing safe concurrent C++ code? [Best Practice]

Answer

Golden rules: (1) Prefer message passing over shared state. (2) Use RAII for all locks. (3) Never hold a lock while calling unknown code. (4) Lock in consistent order everywhere. (5) Minimize critical section size. (6) Prefer higher-level abstractions (futures, channels) over raw mutexes. (7) Use atomic only when you need it and understand the memory model. (8) Document thread ownership and lifetime of shared objects. (9) Run with TSan regularly.

Example

```
// RULE 1: prefer message passing
ThreadSafeQueue<Message> channel;
// Thread A: channel.push(msg);
// Thread B: channel.pop(msg);
// No shared STATE, no locks needed in business logic

// RULE 2: RAII locks always
{ std::lock_guard lk(m); modify(); } // never raw lock/unlock

// RULE 3: no unknown calls under lock
std::function<void()> cb;
{ std::lock_guard lk(m); cb = callback_; } // copy under lock
if (cb) cb(); // call OUTSIDE lock

// RULE 4: consistent lock order
// Document: always lock m1 before m2
std::scoped_lock lk(m1, m2); // enforces order

// RULE 5: short critical sections
{ std::lock_guard lk(m); data = snapshot(); }
processSnapshot(data); // outside lock
```

Q58. What is the impact of cache coherence on multi-threaded performance? [Best Practice]
[Performance]**Answer**

Modern CPUs have multiple cores, each with private L1/L2 caches. The MESI protocol keeps caches coherent: when one core writes a cache line, other cores' copies are invalidated. This inter-core communication costs ~50-200 cycles. Heavily shared atomic variables become bottlenecks. Solutions: (1) Thread-local accumulators merged at end. (2) Padded structures to prevent false sharing. (3) Read-mostly data in shared_mutex. (4) Minimize cross-thread sharing.

Example

```
// SLOW: all threads hammer same atomic -- cache ping-pong
std::atomic<long> globalCounter{0};
void worker() {
    for (int i = 0; i < 1000000; ++i)
        globalCounter.fetch_add(1); // ~50 cycles each!
}

// FAST: thread-local accumulation
thread_local long localCounter = 0;
void workerFast() {
    for (int i = 0; i < 1000000; ++i)
        localCounter++; // ~1 cycle, no sharing
}
// Merge at end: O(threads) atomic ops total

// MESI states: Modified, Exclusive, Shared, Invalid
// When T1 writes: T2's copy -> Invalid (costly IPI)

// Benchmark: thread-local is ~100x faster than
// global atomic for independent per-thread counts
```

Q59. How do you profile and diagnose concurrency bottlenecks? [Best Practice]**Answer**

Tools: (1) perf stat / perf record on Linux — shows cache misses, context switches, CPU cycles. (2) Intel VTune / AMD uProf — cache line contention analysis. (3) Valgrind/Helgrind — lock analysis, deadlock detection. (4) std::chrono benchmarks comparing single vs multi-threaded throughput. (5) Mutex contention metrics by counting lock wait times. Indicators of contention: CPU usage < N*100% with N threads, high sys% time, many voluntary context switches.

Example

```
// Measure lock contention manually
class InstrumentedMutex {
    std::mutex m_;
    std::atomic<uint64_t> contentions_{0};
    std::atomic<uint64_t> waitNs_{0};
public:
    void lock() {
        if (!m_.try_lock()) {
            ++contentions_;
            auto t0 = std::chrono::high_resolution_clock::now();
            m_.lock();
            auto dt = std::chrono::high_resolution_clock::now() - t0;
            waitNs_ += dt.count();
        }
    }
    void unlock() { m_.unlock(); }
    uint64_t contentions() const { return contentions_; }
    double avgWaitMs() const {
```

```

        return contentions_ ?
            waitNs_.load()*1e-6/contentions_ : 0;
    }
};

```

Q60. What does `std::atomic_ref` (C++20) provide? [Best Practice] [C++20]

Answer

`std::atomic_ref<T>` provides atomic operations on an existing non-atomic object. The underlying object must be trivially copyable and suitably aligned. Useful for: making a single field of a struct atomic without making the whole struct atomic, interfacing with C APIs that give you a raw pointer to data you want to access atomically, and legacy code where you cannot change the type.

Example

```

#include <atomic>

// Non-atomic struct
struct Packet {
    int id;
    int seqNum; // we want this to be atomically updated
    char data[256];
};

Packet pkt{};

// Atomic reference to one field
std::atomic_ref<int> atomicSeq(pkt.seqNum);
atomicSeq.fetch_add(1, std::memory_order_relaxed);

// C API: gives us int*
int* rawPtr = getSharedCounter();
std::atomic_ref<int> atomicRaw(*rawPtr);
atomicRaw.fetch_add(1);

// Constraint: alignment must match atomic requirements
static_assert(sizeof(int) <= 8); // platform check

// Multiple atomic_refs to same object are safe
std::atomic_ref<int> ref1(pkt.seqNum);
std::atomic_ref<int> ref2(pkt.seqNum); // OK

```

SECTION 10 — Expert & Interview-Level Questions

Q61. Implement a lock-free MPMC (multi-producer multi-consumer) queue. [Expert]

Answer

An MPMC queue must handle concurrent producers AND consumers. A ring buffer with atomic head and tail using CAS allows multiple producers to claim slots and multiple consumers to claim items. Each slot has a sequence number to avoid the ABA problem. This is the foundation of high-performance queues in Disruptor, LMAX, and trading systems.

Example

```

template<typename T, size_t Cap>

```

```

class MPMCQueue {
    struct Slot {
        std::atomic<size_t> seq;
        T data;
    };
    alignas(64) std::array<Slot, Cap> buf_;
    alignas(64) std::atomic<size_t> head_{0};
    alignas(64) std::atomic<size_t> tail_{0};
public:
    MPMCQueue() {
        for (size_t i = 0; i < Cap; ++i)
            buf_[i].seq.store(i, std::memory_order_relaxed);
    }

    bool push(T val) {
        size_t pos = tail_.load(std::memory_order_relaxed);
        while (true) {
            auto& slot = buf_[pos % Cap];
            size_t seq = slot.seq.load(std::memory_order_acquire);
            intptr_t diff = (intptr_t)seq - (intptr_t)pos;
            if (diff == 0) {
                if (tail_.compare_exchange_weak(pos, pos+1,
                    std::memory_order_relaxed)) {
                    slot.data = std::move(val);
                    slot.seq.store(pos+1, std::memory_order_release);
                    return true;
                }
            } else if (diff < 0) return false; // full
            else pos = tail_.load(std::memory_order_relaxed);
        }
    }

    bool pop(T& out) {
        size_t pos = head_.load(std::memory_order_relaxed);
        while (true) {
            auto& slot = buf_[pos % Cap];
            size_t seq = slot.seq.load(std::memory_order_acquire);
            intptr_t diff = (intptr_t)seq - (intptr_t)(pos+1);
            if (diff == 0) {
                if (head_.compare_exchange_weak(pos, pos+1,
                    std::memory_order_relaxed)) {
                    out = std::move(slot.data);
                    slot.seq.store(pos+Cap, std::memory_order_release);
                    return true;
                }
            } else if (diff < 0) return false; // empty
            else pos = head_.load(std::memory_order_relaxed);
        }
    }
};

```

Q62. What is the Disruptor pattern and how does it achieve ultra-low latency? [Expert]

Answer

The Disruptor (LMAX) is a lock-free MPMC queue using a ring buffer with power-of-2 size. Key innovations: (1) No garbage (pre-allocated slots). (2) Cache-line padded sequence numbers to prevent false sharing. (3) Batching: consumers process multiple items per iteration. (4) Wait strategies: busy-spin for lowest latency, yield/sleep for lower CPU. Used in electronic trading for sub-microsecond order processing.

Example

```
// Disruptor design principles:
```

```
// 1. Ring buffer: pre-allocated, no GC
```

```
constexpr size_t RING_SIZE = 1 << 17; // power of 2
alignas(64) Event ringBuffer[RING_SIZE]; // pre-allocated

// 2. Sequence numbers (not pointers) avoid ABA
struct alignas(64) Sequence {
    std::atomic<long> value{-1};
    char pad[64 - sizeof(std::atomic<long>)];
};

// 3. Producer claims slots with CAS
Sequence producerSeq;
long claimSlot() {
    return producerSeq.value.fetch_add(1,
        std::memory_order_relaxed);
}

// 4. Consumer spins on sequence (busy-wait for lowest latency)
Sequence consumerSeq;
void consumeAll() {
    long nextToRead = consumerSeq.value.load() + 1;
    while (nextToRead <= producerSeq.value.load(
        std::memory_order_acquire)) {
        process(ringBuffer[nextToRead & (RING_SIZE-1)]);
        consumerSeq.value.store(nextToRead++,
            std::memory_order_release);
    }
}

// Result: < 1 microsecond end-to-end latency
```

Q63. What is hazard pointer and how does it solve the memory reclamation problem in lock-free code? [Expert]

Answer

In lock-free structures, a thread may read a pointer, another thread removes and frees the object, and the first thread dereferences freed memory. Hazard pointers: before dereferencing a pointer, a thread publishes it as "hazardous." The deleter checks all hazard pointers before freeing — if any thread has published the pointer, deferring deletion. After the reader is done, it clears the hazard pointer. This gives safe memory reclamation without garbage collection.

Example

```
// Hazard pointer concept
thread_local void* hazardPtr = nullptr; // "I am using this"

// Reader:
void* readSafe(std::atomic<void*>& sharedPtr) {
    void* ptr;
    do {
        ptr = sharedPtr.load(std::memory_order_acquire);
        hazardPtr = ptr; // publish: do not free this!
    } while (ptr != sharedPtr.load(std::memory_order_seq_cst));
    return ptr; // safe to dereference
}

void releaseHazard() {
    hazardPtr = nullptr; // clear: safe to free now
}

// Writer/deleter:
void retire(void* ptr) {
    // Scan all hazard pointers
    for (auto hp : allHazardPointers) {
```

```

        if (hp == ptr) {
            deferredFree.push_back(ptr); // not yet safe
            return;
        }
        ::operator delete(ptr); // safe to free
    }
}

// C++26 will standardize hazard pointers: std::hazard_pointer

```

Q64. What is RCU (Read-Copy-Update) and when is it superior to reader-writer locks? [Expert]

Answer

RCU allows reads to be completely lock-free and wait-free — readers never block and never contend with each other. Writers create new versions and wait for all pre-existing readers to finish before reclaiming old data. RCU is superior to `shared_mutex` when: reads vastly outnumber writes, read latency is critical, reader starvation is unacceptable. Downside: writer must wait for all current readers (grace period), and memory usage is higher (old version kept during grace period).

Example

```

// Production RCU with quiescent state tracking
class RCUManager {
    std::atomic<uint64_t> globalEpoch_{0};
    thread_local static uint64_t localEpoch_;
    thread_local static bool inCS_; // in critical section

    std::vector<std::atomic<uint64_t>*> workerEpochs_;
public:
    // Reader: publish entry epoch
    void readLock() {
        inCS_ = true;
        localEpoch_ = globalEpoch_.load(std::memory_order_acquire);
    }
    void readUnlock() { inCS_ = false; }

    // Writer: wait for all readers in old epoch
    void synchronize() {
        uint64_t target = globalEpoch_.fetch_add(1) + 1;
        // Wait until all workers have seen new epoch
        for (auto* epoch : workerEpochs_) {
            while (epoch->load() < target && inCS_)
                std::this_thread::yield();
        }
        // Now safe to free old data
    }
};

```

Q65. How does a concurrent skip list work and what are its advantages? [Expert]

Answer

A concurrent skip list is a sorted lock-free (or fine-grained lock) data structure providing $O(\log n)$ average-case insert, delete, and search. Nodes have multiple forward pointers at different levels. Fine-grained locking: lock only the affected tower of pointers. Lock-free: use CAS to link/unlink nodes with a logical deletion flag (marked bit in the next pointer). Used in Java's `ConcurrentSkipListMap`, Intel TBB.

Example

```

// Lock-free skip list node
template<typename K, typename V>
struct SkipNode {

```

```

    K key; V val;
    std::atomic<SkipNode*> next[MAX_LEVEL];
    // "Marked" bit in LSB of next pointer = logically deleted
};

// Logical deletion: mark next pointer before physical removal
SkipNode* markPtr(SkipNode* p) {
    return reinterpret_cast<SkipNode*>(
        reinterpret_cast<uintptr_t>(p) | 1);
}

bool isMarked(SkipNode* p) {
    return reinterpret_cast<uintptr_t>(p) & 1;
}

SkipNode* getPtr(SkipNode* p) {
    return reinterpret_cast<SkipNode*>(
        reinterpret_cast<uintptr_t>(p) & ~1);
}

// Deletion: mark all levels, then physically unlink
// Search: skip marked nodes
// Insert: CAS to link new node at bottom level first

// Advantage over balanced BST:
// - Easier lock-free implementation
// - No rebalancing (rotations) required

```

Q66. What is the Epoch-Based Reclamation (EBR) scheme? [Expert]

▶ Answer

EBR is a memory reclamation scheme for lock-free data structures, simpler than hazard pointers. There are 3 epochs (0,1,2). Threads announce their current epoch when entering a critical section. Writers retire objects to the current epoch's limbo list. When all threads have moved past an epoch, all objects retired in that epoch are safe to free. EBR has lower overhead than hazard pointers but can delay reclamation if a thread stalls.

Example

```

// EBR: 3-epoch scheme
std::atomic<int> globalEpoch{0}; // cycles 0,1,2,0,1,2...
thread_local int localEpoch = -1;
thread_local std::vector<void*> limboBag[3]; // per-epoch

// Enter critical section: announce epoch
void enterCS() {
    localEpoch = globalEpoch.load(std::memory_order_acquire);
}

// Exit critical section
void exitCS() { localEpoch = -1; }

// Retire: add to current epoch's limbo bag
void retire(void* ptr) {
    int epoch = globalEpoch.load();
    limboBag[epoch].push_back(ptr);
}

// Try advance: if all threads in current or newer epoch,
// free oldest epoch's bag and advance global epoch
void tryAdvance() {
    int cur = globalEpoch.load();
    for (auto& e : allThreadEpochs)
        if (e != -1 && e != cur) return; // straggler
}

```

```
// Safe to free (cur+1)%3's limbo bag
for (auto* p : limboBag[(cur+1)%3]) ::operator delete(p);
limboBag[(cur+1)%3].clear();
globalEpoch.store((cur+1)%3, std::memory_order_release);
}
```

Q67. How do you design a lock-free order book for electronic trading? [Expert] [Trading]

Answer

A trading order book must process millions of updates per second with microsecond latency. Key design: (1) Per-price-level atomic linked lists or arrays. (2) Lock-free SPSC queue from the network feed handler to the order book. (3) Single-writer ownership: only the book thread modifies bids/asks. (4) Atomic snapshots via RCU for risk/reporting consumers. (5) Pre-allocated object pools. (6) Avoid any heap allocation on the hot path.

Example

```
// Ultra-low latency order book design

// Price level: intrusive linked list of orders
struct Order {
    int64_t price;      // fixed-point price
    int64_t quantity;   // shares
    uint64_t orderId;
    Order* next = nullptr; // intrusive list
};

// Single-writer book: only book thread modifies
class OrderBook {
    std::map<int64_t, Order*> bids_; // price->orders
    std::map<int64_t, Order*> asks_;
    ObjectPool<Order> pool_; // no heap on hot path

    // RCU snapshot for consumers
    std::atomic<BookSnapshot*> snapshot_;
public:
    // Called ONLY from single book thread
    void addOrder(const Message& msg) {
        Order* o = pool_.allocate(); // O(1), no malloc
        *o = {msg.price, msg.qty, msg.id};
        auto& side = msg.isBid ? bids_ : asks_;
        o->next = side[msg.price];
        side[msg.price] = o; // no lock needed: single writer
        publishSnapshot(); // RCU update for consumers
    }
};

// Feed: SPSC queue from network -> book thread
SPSCQueue<Message, 65536> feedQueue;
```

Q68. What is the difference between coarse-grained and fine-grained locking in a hash table? [Expert]

Answer

Coarse-grained: one lock for the whole table — simple but all operations serialize. Fine-grained: one lock per bucket — threads on different buckets proceed in parallel. Strip locking: one lock per N buckets (fewer mutexes). Lock-free: no locks at all using CAS on bucket lists. The trade-off is implementation complexity vs. scalability. Fine-grained scales with the number of buckets but has higher per-operation overhead.

Example

```
// Coarse-grained: simple, poor parallelism
```

```

class CoarseMap {
    std::unordered_map<int,int> map_;
    std::mutex m_;
public:
    void put(int k, int v) { std::lock_guard lk(m_); map_[k]=v; }
    int get(int k)         { std::lock_guard lk(m_); return map_.at(k); }
};

// Fine-grained: N bucket mutexes
class FineMap {
    static constexpr int N = 64;
    struct Bucket {
        std::unordered_map<int,int> map;
        mutable std::shared_mutex mtx;
    };
    std::array<Bucket, N> buckets_;
    Bucket& getBucket(int k) { return buckets_[k%N]; }
public:
    void put(int k, int v) {
        auto& b = getBucket(k);
        std::unique_lock lk(b.mtx);
        b.map[k] = v; // only this bucket locked
    }
    int get(int k) const {
        auto& b = getBucket(k);
        std::shared_lock lk(b.mtx); // readers parallel
        return b.map.at(k);
    }
};
// FineMap: N threads can read/write different buckets simultaneously

```

Q69. How does `std::execution` (C++17 parallel algorithms) work internally? [Expert]

Answer

Parallel algorithms with execution policies are backed by a thread pool (implementation-defined — often TBB or a vendor pool). The algorithm divides the work range into chunks and submits them to the pool. For `par`: independent sub-ranges sorted/transformed in parallel, then merged. For `par_unseq`: additionally uses SIMD vectorization. The standard gives no guarantee on thread count or scheduling — it is purely a hint.

Example

```

#include <execution>
#include <algorithm>
#include <numeric>

std::vector<double> v(10000000);
std::iota(v.begin(), v.end(), 0.0);

// Parallel sort: O(n log n) on N cores
std::sort(std::execution::par, v.begin(), v.end());

// Parallel sum: reduce in parallel then combine
double sum = std::reduce(std::execution::par_unseq,
    v.begin(), v.end(), 0.0);

// Parallel transform_reduce: map then reduce
double norm = std::transform_reduce(
    std::execution::par,
    v.begin(), v.end(), 0.0,
    std::plus<double>(),
    [](double x){ return x*x; });

// Internal: TBB splits range recursively,
// submits to work-stealing pool,

```

```
// uses SIMD for par_unseq

// Requirements for user functions:
// - Thread-safe (par)
// - No ordering dependencies between elements
```

Q70. What concurrency features are planned/added in C++23 and C++26? [Expert] [C++23]

Answer

C++23 adds: `std::generator` (stackless coroutine ranges), `std::execution` (P2300 senders/receivers — a composable async programming model replacing futures). C++26 targets: `std::hazard_pointer` (safe memory reclamation), `std::rcu` (read-copy-update), `std::synchronized_value` (mutex + value combined), `atomic_ref` improvements, and improved coroutine support. The P2300 execution model is the most significant: structured concurrency, composable async algorithms, cancellation.

Example

```
// C++23: std::generator
#include <generator>
std::generator<int> fibonacci() {
    int a=0, b=1;
    while (true) { co_yield a; auto c=a+b; a=b; b=c; }
}

// C++23: P2300 std::execution (senders/receivers)
// Structured async: composable, cancellable, efficient

// Sender: describes work to be done
// Receiver: handles result/error/cancellation
// Scheduler: where work runs

// Example (P2300 syntax):
// auto work = ex::when_all(
//     ex::on(pool, task1()),
//     ex::on(pool, task2()))
// | ex::then([](auto r1, auto r2){ combine(r1,r2); });
// ex::sync_wait(work);

// C++26: std::hazard_pointer
// auto hp = std::make_hazard_pointer(domain);
// hp.protect(atomicPtr); // publish hazard
// // ... use ptr ...
// hp.reset_protection(); // clear hazard
```

Q71. What is transactional memory (STM/HTM) and how does it relate to C++? [Expert]

Answer

Transactional Memory (TM) allows regions of code to execute atomically and in isolation, like database transactions, without explicit locks. Hardware TM (HTM, Intel TSX): CPU tracks reads/writes in L1 cache, commits atomically if no conflict, aborts on conflict/overflow. Software TM (STM): runtime tracks accesses, slower. C++ had a TM TS (Technical Specification) but it was never standardized. Intel TSX was disabled on many CPUs due to security issues (TAA).

Example

```
// C++ TM TS syntax (not standardized)
// __transaction_atomic {
//     x += 1; y -= 1; // atomic block
// }
```

```
// Intel TSX intrinsics (hardware)
#include <immintrin.h>

int status = _xbegin();
if (status == _XBEGIN_STARTED) {
    // Transactional execution
    x += 1; y -= 1;
    _xend(); // commit -- atomic if no conflict
} else {
    // Fallback: lock-based (TSX aborted)
    std::lock_guard lk(mutex);
    x += 1; y -= 1;
}

// TSX abort reasons:
// - Conflict: another thread touched our read/write set
// - Overflow: L1 cache too small
// - System call inside transaction
// - Interrupt / debug breakpoint

// TSX disabled on many CPUs after Microarchitectural
// Data Sampling (MDS) vulnerabilities 2019-2020
```

Q72. How do you implement a high-frequency trading feed handler? [\[Expert\]](#) [\[Trading\]](#)

Answer

A feed handler receives market data (UDP multicast or TCP), parses messages, and updates the order book. Critical design choices: (1) Busy-polling (recvmsg in tight loop, no blocking). (2) CPU pinning and NUMA awareness. (3) Lock-free SPSC queue to book thread. (4) Pre-allocated message objects. (5) Kernel bypass (DPDK, Solarflare OpenOnload) to eliminate OS overhead. Latency target: < 1 microsecond from packet receipt to book update.

Example

```
// Feed handler architecture

// 1. Network thread: busy-polls NIC
void networkThread(int sockFd) {
    cpu_set_t cpus; CPU_ZERO(&cpus);
    CPU_SET(0, &cpus); // pin to core 0
    pthread_setaffinity_np(pthread_self(), sizeof(cpus), &cpus);

    alignas(64) char buf[2048];
    while (running) {
        ssize_t n = recv(sockFd, buf, sizeof(buf),
                        MSG_DONTWAIT); // non-blocking!

        if (n > 0) {
            auto* msg = msgPool.allocate(); // no malloc
            parseMessage(buf, n, msg);
            spscQueue.push(msg); // lock-free push
        }
        // No sleep! Busy-poll for minimum latency
    }
}

// 2. Book thread: processes messages
void bookThread() {
    cpu_set_t cpus; CPU_ZERO(&cpus);
    CPU_SET(1, &cpus); // pin to core 1
    pthread_setaffinity_np(pthread_self(), sizeof(cpus), &cpus);

    Message* msg;
    while (running) {
        if (spscQueue.pop(msg)) {
```

```

        orderBook.apply(*msg); // single writer
        msgPool.release(msg);
    }
}

```

Q73. What is the C++ memory model's relationship to the hardware memory model? [Expert]

Answer

The C++ abstract machine defines a portable memory model. Different hardware has different guarantees: x86/x64 uses Total Store Order (TSO) — all writes become globally visible in program order; only StoreLoad reordering is possible. ARM/POWER are weakly ordered — any reordering is possible unless fences are used. C++ memory_order maps to hardware fences: seq_cst -> full memory fence (mfence on x86, dmb ish on ARM); release/acquire -> often free on x86 (TSO) but fence on ARM.

Example

```

// On x86 (TSO): these are equivalent for release/acquire
// (TSO makes all stores visible in order -- no fence needed)
atomic.store(1, std::memory_order_release);
// x86 asm: MOV [addr], 1 (no fence!)

// On ARM: requires explicit store-release fence
// ARM asm: STLR r1, [addr] (store with release semantics)

// seq_cst always needs a fence on x86:
atomic.store(1, std::memory_order_seq_cst);
// x86 asm: MOV [addr], 1; MFENCE (full barrier)

// relaxed: no fence on any architecture
atomic.store(1, std::memory_order_relaxed);
// x86 asm: MOV [addr], 1

// Implication: relaxed vs acquire/release has NO cost on x86
// but significant cost on ARM/POWER
// -> portability argument for using correct memory_order
// even when targeting x86 only

```

Q74. How do you implement a concurrent reference-counted object without shared_ptr overhead? [Expert]

Answer

std::shared_ptr has two indirections (pointer to control block, pointer to object) and two atomic operations per copy. For hot paths, intrusive reference counting (count inside the object) is faster: one pointer, one atomic op per copy, better cache locality. The object inherits from a ref-counted base. Use when shared_ptr overhead is measurable — typically < 1% of programs need this optimization.

Example

```

// Intrusive ref-count base
class RefBase {
    mutable std::atomic<int> count_{1};
public:
    void addRef() const noexcept {
        count_.fetch_add(1, std::memory_order_relaxed);
    }
    void release() const noexcept {
        if (count_.fetch_sub(1, std::memory_order_acq_rel) == 1)
            delete this;
    }
}

```

```
protected:
    virtual ~RefBase() = default;
};

// RAII intrusive pointer
template<typename T>
class IntrusivePtr {
    T* p_ = nullptr;
public:
    explicit IntrusivePtr(T* p) : p_(p) {} // takes ownership
    ~IntrusivePtr() { if (p_) p_>release(); }
    IntrusivePtr(const IntrusivePtr& o) : p_(o.p_)
    { if (p_) p_>addRef(); }
    IntrusivePtr(IntrusivePtr&& o) noexcept
    : p_(std::exchange(o.p_, nullptr)) {}
    T* operator->() const { return p_; }
    T& operator*() const { return *p_; }
};

class MyObj : public RefBase { /* ... */ };
IntrusivePtr<MyObj> obj(new MyObj());
```

Q75. What are the NUMA (Non-Uniform Memory Access) considerations in multi-threaded code? [Expert]

▶ Answer

On multi-socket systems, memory access time depends on which NUMA node the memory is on. A thread on CPU 0 accessing memory allocated on NUMA node 1 may take 2-3x longer than local access. Best practices: (1) Allocate memory on the NUMA node where it will be used (`numa_alloc_local`). (2) Pin threads to specific NUMA nodes. (3) Avoid cross-NUMA sharing. (4) Use thread-local storage. (5) Use `libnuma` or `hwloc` for NUMA-aware allocation.

Example

```
#include <numa.h>

// Check NUMA availability
if (numa_available() >= 0) {
    int numNodes = numa_max_node() + 1;
    std::cout << "NUMA nodes: " << numNodes << "\n";
}

// Pin thread to NUMA node 0
void pinToNode(int node) {
    struct bitmask* mask = numa_allocate_nodemask();
    numa_bitmask_setbit(mask, node);
    numa_bind(mask);
    numa_free_nodemask(mask);
}

// Allocate memory local to current thread's NUMA node
void* localMem = numa_alloc_local(size);
// vs remote (slow):
// void* remoteMem = numa_alloc_onnode(size, otherNode);

// C++: use PMR with NUMA-aware allocator
struct NumaResource : std::pmr::memory_resource {
    void* do_allocate(size_t n, size_t) override {
        return numa_alloc_local(n);
    }
    void do_deallocate(void* p, size_t n, size_t) override {
        numa_free(p, n);
    }
}
```

```
bool do_is_equal(const memory_resource& o) const noexcept override
{ return &o == this; }
};
```

Q76. How do you implement a scalable readers-writers lock without writer starvation? [Expert]

Answer

A simple readers-writers lock can starve writers if readers keep arriving. To prevent starvation: once a writer is waiting, new readers must queue behind it. This is a "writer-preference" or "fair" RW lock. Implemented with a mutex + two condition variables + a "writer waiting" flag that new readers check before entering the read side.

Example

```
class FairRWLock {
    std::mutex m_;
    std::condition_variable readers_cv_, writers_cv_;
    int readers_ = 0;
    bool writerActive_ = false;
    int writersWaiting_ = 0;
public:
    void readLock() {
        std::unique_lock lk(m_);
        // Wait if writer active OR writer waiting (fairness)
        readers_cv_.wait(lk, [this]{
            return !writerActive_ && writersWaiting_ == 0;
        });
        ++readers_;
    }

    void readUnlock() {
        std::unique_lock lk(m_);
        if (--readers_ == 0)
            writers_cv_.notify_one(); // wake waiting writer
    }

    void writeLock() {
        std::unique_lock lk(m_);
        ++writersWaiting_;
        writers_cv_.wait(lk, [this]{
            return !writerActive_ && readers_ == 0;
        });
        --writersWaiting_;
        writerActive_ = true;
    }

    void writeUnlock() {
        std::unique_lock lk(m_);
        writerActive_ = false;
        if (writersWaiting_) writers_cv_.notify_one();
        else readers_cv_.notify_all();
    }
};
```

Q77. What is a futex and how do high-performance mutexes use it? [Expert]

Answer

A futex (Fast Userspace muTEX) is a Linux kernel primitive that is the foundation of all modern mutexes.

Uncontended case: mutex state is managed purely in userspace with an atomic CAS — no system call, ~5ns.

Contended case: the losing thread calls futex(FUTEX_WAIT) to sleep in the kernel. The releasing thread calls

futex(FUTEX_WAKE) to wake a waiter. This makes uncontended locking nearly free. `std::mutex` on Linux is implemented with futex.

Example

```
#include <linux/futex.h>
#include <sys/syscall.h>

// Simplified futex mutex
class FutexMutex {
    // 0 = unlocked, 1 = locked, 2 = locked+waiters
    std::atomic<int> state_{0};
public:
    void lock() {
        int expected = 0;
        // Fast path: CAS from 0 to 1 -- no syscall!
        if (state_.compare_exchange_strong(expected, 1,
            std::memory_order_acquire)) return;

        // Slow path: set state=2, sleep in kernel
        while (state_.exchange(2,
            std::memory_order_acquire) != 0) {
            syscall(SYS_futex, &state_,
                FUTEX_WAIT, 2, nullptr, nullptr, 0);
        }
    }

    void unlock() {
        // Fast path: if state was 1 (no waiters), done
        if (state_.fetch_sub(1, std::memory_order_release) != 1) {
            state_.store(0, std::memory_order_release);
            // Wake one waiter
            syscall(SYS_futex, &state_,
                FUTEX_WAKE, 1, nullptr, nullptr, 0);
        }
    }
};
```

Q78. What are spinlocks and when are they appropriate? [Expert]

Answer

A spinlock busy-waits (spins in a loop) rather than blocking the thread. Appropriate when: (1) Critical section is very short (< 100 instructions). (2) Thread cannot sleep (interrupt context, real-time thread). (3) Contention is very low. Inappropriate when: long critical sections (wastes CPU), many threads, or system may preempt the lock-holder (priority inversion). Always include a yield or PAUSE instruction in the spin loop to reduce power and memory bus traffic.

Example

```
class SpinLock {
    std::atomic_flag flag_ = ATOMIC_FLAG_INIT;
public:
    void lock() noexcept {
        while (flag_.test_and_set(std::memory_order_acquire)) {
            // Spin -- but be polite to the CPU
#ifdef __x86_64__
            __builtin_ia32_pause(); // x86 PAUSE instruction
#else
            std::this_thread::yield();
#endif
        }
    }
    void unlock() noexcept {
        flag_.clear(std::memory_order_release);
    }
};
```

```
};

// RAII usage
SpinLock sl;
{ std::lock_guard<SpinLock> lk(sl); /* < 100 ns work */ }

// Exponential backoff spinlock (reduces contention)
void lockWithBackoff() {
    for (int pause=1; ; pause=std::min(pause*2,1024)) {
        if (!flag_.test_and_set(std::memory_order_acquire)) break;
        for (int i=0; i<pause; ++i) __builtin_ia32_pause();
    }
}
```

Q79. How do you implement a thread-safe singleton using modern C++? [Expert]

Answer

Three patterns: (1) Meyers Singleton — local static: guaranteed thread-safe initialization in C++11 (no lock needed). (2) call_once: explicitly thread-safe. (3) Atomic DCL: correct with acquire/release semantics. The Meyers Singleton is the simplest and most recommended approach. Never use global static initialization for thread-safe singletons.

Example

```
// BEST: Meyers Singleton -- thread-safe by standard (C++11)
class Config {
    Config() { load(); }
public:
    static Config& instance() {
        static Config inst; // guaranteed once, thread-safe
        return inst;
    }
    Config(const Config&) = delete;
    Config& operator=(const Config&) = delete;
};

// call_once singleton
class Logger {
    static std::once_flag flag_;
    static std::unique_ptr<Logger> inst_;
    Logger() {}
public:
    static Logger& instance() {
        std::call_once(flag_, []{ inst_.reset(new Logger()); });
        return *inst_;
    }
};

// Testable: reset via call_once not possible
// For testable singletons: use dependency injection instead
class App {
    Logger& logger_; // injected, not singleton
public:
    App(Logger& l) : logger_(l) {}
};
```

Q80. What is instruction-level parallelism and how does it affect concurrent code? [Expert]

Answer

Modern CPUs execute multiple instructions simultaneously (superscalar, out-of-order execution). Memory barriers (fences) limit this by constraining reordering. Atomic operations with `seq_cst` issue full fences, serializing the pipeline. This is why `seq_cst` is expensive and `relaxed` is cheap — `relaxed` allows maximum ILP. Understanding this helps write correct, high-performance lock-free code.

Example

```
// Full fence: serializes everything
std::atomic<int> x{0};
x.store(1, std::memory_order_seq_cst);
// x86 asm: mov [x], 1
//          mfence          <-- drains pipeline

// Release: only prevents StoreStore reorder
x.store(1, std::memory_order_release);
// x86 asm: mov [x], 1      <-- cheaper, TSO guarantees enough

// Relaxed: no fence, max ILP
x.store(1, std::memory_order_relaxed);
// x86 asm: mov [x], 1      <-- same on x86, cheaper on ARM

// Thread-local counters: maximum ILP, no atomic needed
thread_local uint64_t msgCount = 0;
void onMessage() { ++msgCount; } // 1 cycle, no fence

// Merge: one atomic add per thread, not per message
std::atomic<uint64_t> totalMsgs{0};
void flushCount() {
    totalMsgs.fetch_add(msgCount, std::memory_order_relaxed);
    msgCount = 0;
}
```

Q81. How do you implement a wait-free counter and why is it significant? [Expert]

Answer

`fetch_add` on an atomic integer is wait-free on x86 (LOCK XADD instruction) and most modern architectures — it completes in bounded time regardless of other threads' actions. This is why per-thread counters that use `fetch_add` to merge are so much better than mutex-based counters. Wait-free counters are the gold standard for high-frequency event counting.

Example

```
// Wait-free: guaranteed to complete in fixed cycles
std::atomic<uint64_t> eventCount{0};

void recordEvent() {
    // x86: LOCK XADD -- one instruction, ~40 cycles
    // Wait-free: completes regardless of other threads
    eventCount.fetch_add(1, std::memory_order_relaxed);
}

// Distributed counter: no cross-thread atomic ops
struct alignas(64) PerThreadCounter {
    uint64_t local = 0;
};

std::vector<PerThreadCounter> perThread;

thread_local int threadIdx; // assigned at thread start

void recordLocal() {
    perThread[threadIdx].local++; // 1 cycle, no atomic
}
```

```
uint64_t total() {
    uint64_t sum = 0;
    for (auto& c : perThread) sum += c.local;
    return sum;
}
```

Q82. What is structured concurrency and why is it important? [Expert] [C++20]

Answer

Structured concurrency ensures that concurrent tasks have a clear lifetime scope — all spawned tasks complete before the spawning scope exits, similar to how structured control flow (loops, if statements) works. This prevents dangling references, simplifies reasoning about lifetime, and makes cancellation composable. C++20 `jthread` + `stop_token` is a step in this direction. P2300 (`std::execution`) in C++23 provides full structured concurrency via sender/receiver model.

Example

```
// Unstructured: threads escape their scope
void badAsync() {
    int local = 42;
    std::thread t([&local]{ use(local); }); // local may die!
    t.detach(); // scope exits, local destroyed, t accesses UB
}

// Structured: scope owns all child tasks
void goodAsync() {
    int local = 42;
    {
        // std::jthread joins in destructor
        std::jthread t([&local]{ use(local); });
    } // t.join() called -- local still alive
} // local destroyed safely after thread done

// P2300 structured concurrency (C++23)
// ex::sync_wait(
//     ex::when_all(
//         ex::on(pool, task1()), // structured scope
//         ex::on(pool, task2())  // all complete before sync_wait returns
//     )
// );

// Benefit: no dangling references, no leaking threads
// Cancellation propagates hierarchically
```

Q83. What is thread local storage (TLS) and what are its performance implications? [Expert]

Answer

Thread-local storage gives each thread its own copy of a variable (`thread_local` keyword). Access is slightly slower than regular variables on some platforms (one extra pointer indirection via the TLS block), but much faster than atomic or mutex-based sharing. TLS constructors run when the thread first accesses the variable. TLS destructors run when the thread exits. Perfect for per-thread caches, random number generators, and allocators.

Example

```
// thread_local: each thread has its own copy
thread_local int threadId = -1;
thread_local std::mt19937 rng{std::random_device{}()}; // per-thread RNG

// Thread-local memory allocator (no synchronization)
thread_local MemoryPool localPool{1024 * 1024}; // 1MB per thread
```

```

void* threadLocalAlloc(size_t n) {
    return localPool.allocate(n); // lock-free!
}

// Thread-local destructor: runs on thread exit
struct ThreadLocalCleanup {
    ~ThreadLocalCleanup() {
        flushLocalCounters(); // merge into global
    }
};
thread_local ThreadLocalCleanup cleanup; // auto destructor

// Performance: 1-3 extra cycles on x86 (fs/gs segment)
// vs shared atomic: 40-200 cycles (cache coherence)

// Dynamic TLS (dlopen): more overhead
// Static TLS (main binary): minimal overhead

```

Q84. How do you design a concurrent object that avoids all locking (lock-free)? [Expert]

▶ Answer

Design principles for lock-free objects: (1) Immutable data structures (create new version, swap pointer). (2) Single-writer ownership (only one thread modifies, others read via atomic snapshot). (3) Append-only structures (CAS on tail). (4) Functional-style updates (transform + CAS). (5) Epoch/hazard-pointer-based reclamation. The complexity cost is high — only worth it for proven bottlenecks.

Example

```

// Lock-free immutable config update
struct Config { std::string server; int port; int timeout; };

class AtomicConfig {
    std::atomic<std::shared_ptr<Config>> config_;
public:
    AtomicConfig(Config c)
        : config_(std::make_shared<Config>(std::move(c))) {}

    // READERS: completely lock-free
    std::shared_ptr<Config> get() const {
        return config_.load(std::memory_order_acquire);
    }

    // WRITER: atomic swap
    void update(Config c) {
        config_.store(std::make_shared<Config>(std::move(c)),
                     std::memory_order_release);
        // Old config freed when last reader drops shared_ptr
    }

    // CAS-based conditional update
    bool updateIf(const Config& expected, Config newCfg) {
        auto cur = config_.load();
        if (cur->server != expected.server) return false;
        auto next = std::make_shared<Config>(std::move(newCfg));
        return config_.compare_exchange_strong(cur, next,
        std::memory_order_acq_rel);
    }
};

```

Q85. What are the memory ordering guarantees on x86/ARM and how do they map to C++ memory_order? [Expert]**▶ Answer**

x86 (TSO): stores are visible in program order; loads are visible in program order; the only allowed reordering is StoreLoad (a later load may execute before an earlier store). This means release/acquire semantics are free on x86 — no extra fences needed. ARM/POWER: weakly ordered — any load/store can be reordered with any other. Every acquire/release needs explicit barrier instructions (LDAR/STLR on ARMv8).

Example

```
// Cost comparison: x86 vs ARM

// relaxed: no cost on EITHER architecture
a.store(1, std::memory_order_relaxed);
// x86: MOV [a], 1 (0 extra cycles)
// ARM: STR r1, [a] (0 extra cycles)

// release/acquire: FREE on x86, costs fence on ARM
a.store(1, std::memory_order_release);
// x86: MOV [a], 1 (no fence: TSO guarantees it)
// ARM: STLR r1, [a] (store-release barrier: ~5 cycles)

a.load(std::memory_order_acquire);
// x86: MOV rax, [a] (no fence)
// ARM: LDAR rax, [a] (load-acquire: ~5 cycles)

// seq_cst: full fence on BOTH
a.store(1, std::memory_order_seq_cst);
// x86: MOV [a], 1; MFENCE (~40 cycles)
// ARM: DMB ISH; STLR r1, [a] (~20 cycles)

// Implication: on x86, release/acquire adds ZERO cost
// vs relaxed -- but code is portable to ARM!
```

Q86. How do you implement a concurrent ring buffer for inter-thread communication? [Expert]**▶ Answer**

A ring buffer is fixed-size, pre-allocated, and circular. For SPSC: two atomic indices (head/tail), producer owns tail, consumer owns head. For MPSC: CAS on tail. For MPMC: sequence-numbered slots (Disruptor variant). The key: modular arithmetic with power-of-2 size (mask = size-1) for O(1) index wrapping. Cache-line padding between head and tail prevents false sharing.

Example

```
template<typename T>
class RingBuffer {
    static constexpr size_t SIZE = 1 << 16; // must be power of 2
    static constexpr size_t MASK = SIZE - 1;

    T buf_[SIZE];
    alignas(64) std::atomic<size_t> writeIdx_{0};
    alignas(64) std::atomic<size_t> readIdx_{0};
    // Separate cache lines: NO false sharing between producer/consumer

public:
    // Producer: O(1), wait-free on empty
    bool write(T val) {
        size_t w = writeIdx_.load(std::memory_order_relaxed);
        size_t r = readIdx_.load(std::memory_order_acquire);
        if ((w - r) >= SIZE) return false; // full
        buf_[w & MASK] = std::move(val);
        writeIdx_.store(w+1, std::memory_order_release);
        return true;
    }
};
```

```

    }

    // Consumer: O(1), wait-free on non-empty
    bool read(T& out) {
        size_t r = readIdx_.load(std::memory_order_relaxed);
        size_t w = writeIdx_.load(std::memory_order_acquire);
        if (r == w) return false; // empty
        out = std::move(buf_[r & MASK]);
        readIdx_.store(r+1, std::memory_order_release);
        return true;
    }
};

```

Q87. What is a concurrent skip list and how does it compare to concurrent hash map? [Expert]

Answer

A concurrent skip list is a sorted, ordered data structure with $O(\log n)$ operations, supporting range queries and ordered iteration. A concurrent hash map is $O(1)$ average but unordered. Use skip list when: you need ordered iteration, range queries, or the ceiling/floor operations. Use hash map when: pure lookup/insert/delete performance matters and order is irrelevant. Java's `ConcurrentSkipListMap` vs `ConcurrentHashMap` illustrate this trade-off.

Example

```

// When to use each:

// Concurrent hash map: O(1) lookups
// - Cache: key -> value
// - Symbol tables
// - Frequency counting
tbb::concurrent_hash_map<std::string, int> symbolTable;

// Concurrent skip list: O(log n), ordered
// - Order book: sorted by price
// - Event queue: sorted by time
// - Leaderboard: sorted by score
tbb::concurrent_unordered_set<int> // no ordered variant in TBB
// Use Java's ConcurrentSkipListMap equivalent or hand-rolled

// Range query on skip list: efficient
// Get all orders between price 100 and 110:
// skipList.lower_bound(100) -> upper_bound(110)

// Range query on hash map: O(n) scan
// Must scan all buckets -- very slow

// Operations summary:
// | Op          | Hash Map | Skip List |
// | Lookup      | O(1)     | O(log n)  |
// | Insert      | O(1)     | O(log n)  |
// | Delete      | O(1)     | O(log n)  |
// | Range       | O(n)     | O(log n+k)|
// | Iteration   | unordered| ordered    |

```

Q88. How do you implement an async task scheduler with dependencies (DAG scheduler)? [Expert]

Answer

A DAG (Directed Acyclic Graph) task scheduler runs tasks with dependency constraints — task B runs only after task A completes. Implementation: each task has an atomic pending count; when a task completes it

decrements all dependent tasks' counts; when a count reaches zero the task is enqueued. Used in game engines (task graphs), build systems, and compute shaders.

Example

```
struct Task {
    std::function<void()> fn;
    std::atomic<int> pending{0}; // dependencies remaining
    std::vector<Task*> dependents; // tasks to notify on complete
};

class DAGScheduler {
    ThreadPool pool_;
    void schedule(Task* t) {
        pool_.submit([this, t]{
            t->fn();
            // Notify all dependent tasks
            for (Task* dep : t->dependents) {
                if (dep->pending.fetch_sub(1) == 1) {
                    schedule(dep); // all deps done, enqueue
                }
            }
        });
    }
public:
    explicit DAGScheduler(size_t n) : pool_(n) {}

    void run(std::vector<Task>& tasks) {
        // Enqueue tasks with no dependencies
        for (auto& t : tasks)
            if (t.pending.load() == 0)
                schedule(&t);
    }
};

// Usage: A -> B, A -> C, B+C -> D
// A.pending=0, B.pending=1, C.pending=1, D.pending=2
// A.dependents={B,C}, B.dependents={D}, C.dependents={D}
```

Q89. What is the "thundering herd" problem in multi-threaded code? [Expert]

Answer

Thundering herd: many threads are sleeping on the same condition. A single event wakes ALL of them (notify_all), but only one can proceed — the rest re-block after wasting CPU waking up, locking, and checking. Solutions: (1) Use notify_one instead of notify_all when only one thread can proceed. (2) Use semaphores to wake exactly N threads. (3) Use separate condition variables for different resources. (4) Stagger wakeups.

Example

```
// THUNDERING HERD: 100 threads all wake, 1 gets work
std::condition_variable cv;
std::queue<Task> tasks;

void worker() {
    while (true) {
        std::unique_lock lk(m_);
        cv.wait(lk, [&]{ return !tasks.empty() || stop_; });
        // 100 threads wake -- 99 re-block immediately!
        if (tasks.empty()) return;
        auto t = tasks.front(); tasks.pop();
        lk.unlock();
        t();
    }
}
```

```

void producer() {
    { std::lock_guard lk(m_); tasks.push(work); }
    cv.notify_all(); // WRONG: wakes all 100!
}

// FIX: notify_one -- wakes exactly one
void fixedProducer() {
    { std::lock_guard lk(m_); tasks.push(work); }
    cv.notify_one(); // only one thread wakes
}

// FIX2: binary_semaphore or counting_semaphore
std::counting_semaphore<100> sem{0};
sem.release(); // wake exactly 1 worker

```

Q90. How do you implement a concurrent B-tree or similar tree structure? [Expert]

Answer

Concurrent B-trees use latch coupling (crabbing): lock parent, lock child, release parent if child won't split/merge. This allows concurrent operations on different subtrees. Modern approaches use optimistic concurrency control (OCC): read without lock, validate, and retry on conflict. Blink-trees add right-sibling pointers to allow readers to recover from concurrent splits without holding locks up the tree.

Example

```

// B-tree latch crabbing (simplified)
struct BNode {
    std::shared_mutex latch;
    std::array<int, ORDER> keys;
    std::array<BNode*, ORDER+1> children;
    bool isLeaf = false;
    int numKeys = 0;
};

// Search: shared latch crabbing (readers don't block each other)
int search(BNode* root, int key) {
    BNode* node = root;
    std::shared_lock parentLk(root->latch);
    while (!node->isLeaf) {
        BNode* child = findChild(node, key);
        std::shared_lock childLk(child->latch);
        parentLk.unlock(); // release parent
        parentLk = std::move(childLk);
        node = child;
    }
    // parentLk holds leaf latch
    return lookupLeaf(node, key);
}

// Insert: exclusive latch crabbing
// Lock parent exclusively; if child is "safe" (won't split),
// release parent and lock child exclusively -- repeat down tree

```

Q91. What is the actor model and how does it map to C++ threads? [Expert]

Answer

The Actor model: each actor is an independent entity with a private mailbox. Actors communicate only via messages — no shared state. Each actor processes messages sequentially (no internal concurrency needed). In C++: each actor is an object with a thread or strand, a message queue, and message handlers. The actor model eliminates shared state and therefore most concurrency bugs.

 Example

```
// Actor base class
class Actor {
    ThreadSafeQueue<std::function<void()>> mailbox_;
    std::jthread thread_;

    void run(std::stop_token st) {
        std::function<void()> msg;
        while (!st.stop_requested()) {
            if (mailbox_.pop(msg)) msg();
        }
    }
public:
    Actor() : thread_([this](std::stop_token st){ run(st); }) {}

    // Send message: non-blocking, async
    template<typename F>
    void send(F&& f) {
        mailbox_.push(std::forward<F>(f));
    }
};

// Account actor: no shared state, no locks
class AccountActor : public Actor {
    double balance_ = 0; // private to this actor
public:
    void deposit(double amt) {
        send([this, amt]{ balance_ += amt; }); // message
    }
    void withdraw(double amt) {
        send([this, amt]{ balance_ -= amt; }); // message
    }
};
```

Q92. How does memory model affect compiler optimizations in multithreaded code? [Expert] **Answer**

The compiler is allowed to reorder, hoist, merge, and eliminate memory operations as long as the single-threaded observable behavior is unchanged. In multi-threaded code without proper synchronization, these optimizations break correctness: hoisting a loop-exit check out of the loop (if no write is visible), eliminating a "redundant" store, or merging reads. Only atomic operations or synchronization primitives tell the compiler where reordering is forbidden.

 Example

```
// Compiler may HOIST this check out of the loop:
bool done = false;
std::thread worker([&]{ doWork(); done = true; });

while (!done) {} // Compiler sees no write to done in loop body
// May compile to: if (!done) while(true); // INFINITE LOOP!

// FIX: atomic -- compiler cannot cache/hoist atomic loads
std::atomic<bool> done2{false};
while (!done2.load(std::memory_order_acquire)) {}

// Compiler may ELIMINATE a "redundant" store:
int x = 0;
x = 1;
x = 2; // compiler: only x=2 matters, eliminate x=1
// If another thread reads x between the writes -- missed!

// FIX: volatile is NOT sufficient for threading
```

```
// (no memory ordering guarantees)
// ONLY std::atomic provides correct ordering

// std::atomic<int> x{0};
// x.store(1); x.store(2); // BOTH stores guaranteed visible
```

Q93. How do you implement a concurrent timer/scheduler? [Expert]

Answer

A timer scheduler runs callbacks at specified times. Implementation: a min-heap of (time, callback) pairs protected by a mutex. A timer thread sleeps until the next timer fires using `wait_until` on a condition variable. New timers are submitted via a queue; the timer thread re-evaluates the sleep duration on each new submission.

Example

```
class TimerScheduler {
    struct TimerEntry {
        std::chrono::steady_clock::time_point time;
        std::function<void()> cb;
        bool operator>(const TimerEntry& o) const { return time>o.time; }
    };
    std::priority_queue<TimerEntry,
        std::vector<TimerEntry>,
        std::greater<TimerEntry>> heap_;
    std::mutex m_;
    std::condition_variable cv_;
    std::jthread thread_;

    void run(std::stop_token st) {
        while (!st.stop_requested()) {
            std::unique_lock lk(m_);
            if (heap_.empty()) {
                cv_.wait(lk, [&]{ return !heap_.empty() ||
                    st.stop_requested(); });
                continue;
            }
            auto next = heap_.top().time;
            if (cv_.wait_until(lk, next, [&]{ return
                heap_.top().time <= std::chrono::steady_clock::now()
                || st.stop_requested(); })) {
                auto entry = heap_.top(); heap_.pop();
                lk.unlock();
                entry.cb(); // fire callback outside lock
            }
        }
    }

public:
    TimerScheduler() : thread_([this](std::stop_token st){ run(st); }) {}
    void scheduleAt(std::chrono::steady_clock::time_point t,
        std::function<void()> cb) {
        { std::lock_guard lk(m_); heap_.push({t, std::move(cb)}); }
        cv_.notify_one();
    }
    void scheduleAfter(std::chrono::milliseconds delay,
        std::function<void()> cb) {
        scheduleAt(std::chrono::steady_clock::now()+delay,
            std::move(cb));
    }
};
```

Q94. What is connection pool design in a multi-threaded server? [Expert]**▶ Answer**

A connection pool maintains a set of pre-created connections (to DB, cache, external service) and lends them to threads on request. Features: bounded pool size (backpressure), idle connection validation, timeout-based waiting, and automatic return via RAII. The pool itself must be thread-safe. The RAII guard returns the connection on scope exit.

Example

```
class ConnectionPool {
    std::queue<Connection*> available_;
    std::mutex m_;
    std::condition_variable cv_;
    std::vector<std::unique_ptr<Connection>> all_;
    size_t maxSize_;

public:
    explicit ConnectionPool(size_t n) : maxSize_(n) {
        for (size_t i = 0; i < n; ++i) {
            all_.push_back(std::make_unique<Connection>());
            available_.push(all_.back().get());
        }
    }

    // RAII lease: returns connection on destruction
    struct Lease {
        Connection* conn; ConnectionPool* pool;
        ~Lease() { pool->returnConn(conn); }
        Connection* operator->() { return conn; }
    };

    Lease acquire(std::chrono::milliseconds timeout={}) {
        std::unique_lock lk(m_);
        bool ok = cv_.wait_for(lk, timeout,
            [this]{ return !available_.empty(); });
        if (!ok) throw std::runtime_error("pool exhausted");
        Connection* c = available_.front(); available_.pop();
        return {c, this};
    }

    void returnConn(Connection* c) {
        { std::lock_guard lk(m_); available_.push(c); }
        cv_.notify_one();
    }
};

// Usage
ConnectionPool pool(10);
{
    auto conn = pool.acquire(std::chrono::ms(100));
    conn->query("SELECT ...");
} // connection returned automatically
```

Q95. What is the difference between task parallelism and data parallelism? [Expert]**▶ Answer**

Task parallelism: different threads execute different tasks (heterogeneous work). Data parallelism: multiple threads execute the SAME operation on different data partitions (SIMD for a single thread, parallel algorithms for multiple threads). C++ supports both: `std::async`/thread pool for task parallelism; `std::execution` parallel algorithms + `std::views` for data parallelism. GPU computing (CUDA/OpenCL) is extreme data parallelism.

Example

```
// TASK PARALLELISM: different work in each thread
auto f1 = std::async(std::launch::async, parseJSON);
auto f2 = std::async(std::launch::async, loadImages);
auto f3 = std::async(std::launch::async, queryDatabase);
auto json = f1.get(); auto imgs = f2.get(); auto db = f3.get();

// DATA PARALLELISM: same work on different data
std::vector<int> data(1000000);
// Same transform on every element, in parallel
std::for_each(std::execution::par_unseq,
    data.begin(), data.end(),
    [](int& x){ x = x * x + 1; });

// Hybrid: parallel pipeline (task + data)
// Stage 1: parse (data-parallel)
// Stage 2: route (task-parallel -- different handlers)
// Stage 3: serialize (data-parallel)

// Choose based on:
// - Work type: heterogeneous -> task, homogeneous -> data
// - Memory access: streaming -> data-parallel friendly
// - Load balancing: irregular work -> work-stealing task pool
```

Q96. How does the Amdahl's Law limit parallel speedup? [Expert]

Answer

Amdahl's Law: if fraction f of a program is parallelizable, maximum speedup with N processors is $1/(1-f + f/N)$. As $N \rightarrow \infty$, max speedup = $1/(1-f)$. Example: 95% parallel, 5% serial -> max speedup = 20x regardless of cores. Gustafson's Law is more optimistic: as problem size grows, the serial fraction often shrinks, allowing near-linear scaling. Implication: always find and minimize the serial bottleneck first.

Example

```
// Amdahl's Law calculation
double amdahl(double parallelFraction, int n) {
    double serial = 1.0 - parallelFraction;
    return 1.0 / (serial + parallelFraction / n);
}

// f=0.90 (90% parallel)
std::cout << amdahl(0.90, 4);    // 3.08x speedup
std::cout << amdahl(0.90, 8);    // 4.71x speedup
std::cout << amdahl(0.90, 64);   // 8.25x speedup
std::cout << amdahl(0.90, 1000); // 9.9x speedup (max ~10x)

// f=0.99 (99% parallel)
std::cout << amdahl(0.99, 64);   // 39x speedup
std::cout << amdahl(0.99, 1000); // 91x speedup

// Lessons:
// 1. Find the serial 1%: critical for scaling
// 2. Profile before parallelizing
// 3. Measure actual speedup: often Amdahl predicts max
// 4. Consider overhead of parallelization itself
```

Q97. What is the difference between coroutines and fibers? [Expert]

Answer

Coroutines (C++20): stackless — suspension point and resume are at specific `co_await` points; the stack frame is gone between suspensions; smaller memory footprint; faster context switch. Fibers: stackful — can suspend

at any point (even inside nested calls) via `setjmp/longjmp` or platform-specific primitives; larger memory per fiber (full stack); slower context switch than coroutines but more powerful. `Boost.Context/Fiber` provides stackful fibers.

Example

```
// COROUTINE: stackless, suspends at co_await only
Task<int> coroutineExample() {
    int a = co_await fetchA(); // stackframe saved at specific point
    int b = co_await fetchB(); // resume here
    co_return a + b;
    // Stack frame: only local vars a,b saved (~few hundred bytes)
}

// FIBER: stackful, can suspend ANYWHERE
// #include <boost/context/fiber.hpp>
// boost::context::fiber fiber([](boost::context::fiber&& self) {
//     int a = expensiveCall(); // suspend INSIDE call!
//     self = std::move(self)(); // yield
//     return std::move(self);
// });

// Comparison:
// Coroutine: ~50-200 bytes overhead, ~10ns switch
// Fiber: 4KB-8MB stack, ~100ns switch

// Use coroutine for: async I/O, generators
// Use fiber for: legacy code requiring suspension mid-function
```

Q98. How do you detect and prevent priority inversion in a real-time C++ system? [Expert]

Answer

Priority inversion in real-time systems can cause deadline misses. Prevention strategies: (1) Priority inheritance: OS dynamically elevates low-priority lock-holder to high-priority waiter's level (pthread `PTHREAD_PRIO_INHERIT`). (2) Priority ceiling: mutex has a static ceiling priority, all lockers inherit it. (3) Avoid blocking operations from high-priority threads. (4) Use lock-free structures on hot paths. (5) Bounded priority queues with aging.

Example

```
// POSIX priority inheritance mutex
pthread_mutex_t pi_mutex;
pthread_mutexattr_t attr;

pthread_mutexattr_init(&attr);
pthread_mutexattr_setprotocol(&attr, PTHREAD_PRIO_INHERIT);
pthread_mutex_init(&pi_mutex, &attr);
// Low-priority thread holding this mutex gets elevated
// to high-priority waiter's priority automatically

// Priority ceiling protocol
pthread_mutexattr_setprotocol(&attr, PTHREAD_PRIO_PROTECT);
pthread_mutexattr_setprioceiling(&attr, MAX_PRIORITY);
// All lockers execute at MAX_PRIORITY ceiling

// C++ abstraction
class RTMutex {
    pthread_mutex_t m_;
public:
    RTMutex() {
        pthread_mutexattr_t a;
        pthread_mutexattr_init(&a);
        pthread_mutexattr_setprotocol(&a, PTHREAD_PRIO_INHERIT);
        pthread_mutex_init(&m_, &a);
    }
};
```

```

}
void lock() { pthread_mutex_lock(&m_); }
void unlock() { pthread_mutex_unlock(&m_); }
~RTMutex() { pthread_mutex_destroy(&m_); }
};

```

Q99. What is a wait queue and how is it different from a condition variable? [Expert]

Answer

A condition variable requires external state (a predicate) and an associated mutex. A wait queue is a lower-level primitive: threads park themselves in a queue; another thread explicitly wakes specific threads by handle. `futex(FUTEX_WAIT)` / `futex(FUTEX_WAKE)` is a hardware wait queue. C++20 `atomic::wait()` and `atomic::notify_one()/notify_all()` provide wait queues on atomic variables — no mutex needed.

Example

```

// C++20: atomic::wait -- wait queue without mutex
std::atomic<int> state{0};

// Consumer: wait until state != 0
std::thread consumer([&]{
    state.wait(0); // blocks until state != 0
    // state.wait(old_val): blocks while state == old_val
    std::cout << "State changed to: " << state.load();
});

// Producer: change state and notify
std::this_thread::sleep_for(std::chrono::ms(100));
state.store(1, std::memory_order_release);
state.notify_one(); // or notify_all()
consumer.join();

// Much lower overhead than condition_variable:
// No mutex required
// Maps to futex on Linux (~0 overhead uncontended)

// Multiple waiters
std::atomic<bool> ready{false};
// 10 threads call: ready.wait(false);
// Then: ready.store(true); ready.notify_all();

```

Q100. Design a complete concurrent market data distribution system. [Expert]

Answer

A market data system receives, normalizes, and distributes market data to multiple subscribers. Components: (1) Network receiver (busy-poll, CPU-pinned). (2) Lock-free SPSC queue to normalizer. (3) Normalizer thread applies transformations. (4) RCU-protected snapshot for subscriber reads. (5) Per-subscriber SPSC queues for push delivery. (6) Thread pool for subscriber callbacks. All hot-path components are lock-free.

Example

```

// Complete architecture

struct MarketData { uint64_t ts; char symbol[8]; int64_t price; };

class MarketDataSystem {
    // Network -> Normalizer: lock-free SPSC
    SPSCQueue<RawMsg, 65536> rawQueue_;

    // Normalized snapshot: RCU for readers

```

```

std::atomic<std::shared_ptr<MarketData>> snapshot_;

// Per-subscriber queues: lock-free SPSC
std::vector<SPSCQueue<MarketData, 1024>*> subQueues_;
std::shared_mutex subMtx_; // protect subQueues_ registration

// Thread pool for subscriber callbacks
ThreadPool callbackPool_{4};

std::jthread netThread_;
std::jthread normThread_;

void networkLoop(std::stop_token st) {
    pinToCPU(0); // dedicated core
    char buf[2048];
    while (!st.stop_requested()) {
        if (recv(fd_, buf, sizeof(buf), MSG_DONTWAIT) > 0) {
            RawMsg* msg = msgPool_.alloc();
            parse(buf, msg);
            rawQueue_.push(msg); // lock-free
        }
    }
}

void normalizeLoop(std::stop_token st) {
    pinToCPU(1); // dedicated core
    RawMsg* raw;
    while (!st.stop_requested()) {
        if (rawQueue_.pop(raw)) {
            auto md = normalize(*raw);
            msgPool_.free(raw);
            // RCU update
            snapshot_.store(std::make_shared<MarketData>(md),
                           std::memory_order_release);
            // Distribute to subscribers
            std::shared_lock lk(subMtx_);
            for (auto* q : subQueues_) q->push(md);
        }
    }
}

public:
    MarketDataSystem()
        : netThread_([this](auto st){ networkLoop(st); }),
          normThread_([this](auto st){ normalizeLoop(st); }) {}

    // Subscriber gets lock-free read access
    std::shared_ptr<MarketData> getSnapshot() const {
        return snapshot_.load(std::memory_order_acquire);
    }
};

```